PropMatch

Property Listing and Management System

Final Project Report | Database Management Systems

Relational (MySQL) and NoSQL (MongoDB) design, implementation and testing

Team

Poojitha T	PES1UG24AM192
Bhuvanashree S	PES1UG24AM235

MySQL 8 | MongoDB | Next.js | 13 Tables | 2 Views | 3 Collections

Databases: propmatch (MySQL) | propmatch_nosql (MongoDB)

Contents

01 Introduction and Problem Statement 4
02 Objectives 4
03 Functional Requirements 5
04 Technology Stack and System Architecture 6
05 Database Choice and Justification 8
06 Entity-Relationship Design 9
07 Relational Schema 11
08 Data Dictionary 14
09 Keys, Constraints and Indexes 19
10 MySQL Implementation 20
11 NoSQL / Vector Database Implementation (MongoDB) 23
12 Important Queries 27
13 Application Screenshots 33
14 Testing Results 38
15 SQL – Database Commands 44
16 NoSQL / Vector Database – Commands 48
17 Evaluation Rubrics – Completion Status 50
18 Advantages, Conclusion and Future Enhancements 51

13	2	3	32	201
MySQL tables	Views	MongoDB collections	Property records	Activity events

All figures in this report come from the project's MySQL database **propmatch**, its MongoDB database **propmatch_nosql**, the Next.js web application, and the verification runs described in Section 14.

Project Details

Item	Details
Project title	PropMatch — Property Listing and Management System
Course	Database Management Systems (project report)
Databases	MySQL 8 (propmatch) and MongoDB (propmatch_nosql)
Application	Next.js (App Router) + React front end, Next.js API routes on Node.js

#	Student name	SRN
1	Poojitha T	PES1UG24AM192
2	Bhuvanashree S	PES1UG24AM235

Abstract

PropMatch is a property listing and management system that connects buyers and renters with real-estate agents. It combines a relational database with a document database: MySQL 8 stores the structured records (users, agent profiles, properties, locations, property types, amenities, images, inquiries, site visits, favourites, price history and login audit) in 13 tables and 2 views, and protects them with 43 declared constraints, three triggers, two transactional stored procedures and a function. MongoDB stores buyer preferences, user-activity events and cached recommendations in three collections, with JSON Schema validators, unique and compound indexes and TTL indexes that expire old data automatically.

A Next.js web application lets buyers browse and filter listings, save favourites, send enquiries, book site visits and view personalised “For you” recommendations that show a match percentage and the reasons behind it, while agents manage listings and follow enquiries from a dashboard. This report documents the objectives, requirements, ER diagram, relational schema, data dictionary, database commands, important queries, architecture, application screenshots and testing results. The database rules were verified with 32 executed tests, all of which passed. Vector search is not part of the current implementation.

1 Introduction and Problem Statement

PropMatch is a property listing and management system designed to connect property buyers and renters with agents. The system organizes property information, location details, property types, amenities, images, inquiries and site visits in a structured relational database, and uses a document database to learn from each buyer's preferences and activity so that it can recommend suitable homes.

The project is a hybrid (polyglot) data system: **MySQL 8** holds the structured, relationship-heavy records and enforces integrity, while **MongoDB** holds flexible buyer-preference records, interaction events and cached recommendations. Both are exposed through a single full-stack web application built with Next.js and Node.js.

Problem statement

Managing property listings using separate records can lead to duplicate information, inconsistent data and difficulty retrieving relevant property details. Buyers and renters also need a convenient way to explore available properties, narrow them down to their budget and preferences, and contact agents.

How PropMatch addresses this

Maintains structured property records, connects related information through database relationships, enforces data quality with constraints and triggers, supports property search and management operations, and personalises discovery through buyer preferences and activity stored in MongoDB.

2 Objectives

01 Store property listings and their details in a structured database.	02 Maintain user, buyer and agent information.
03 Organize properties by location and property type.	04 Store property images and amenities.
05 Maintain buyer inquiries and site-visit records.	06 Track property price changes automatically.
07 Support efficient retrieval, filtering and sorting of property information.	08 Apply database constraints to improve data consistency and integrity.
09 Record buyer preferences and activity in a flexible document database.	10 Generate personalised, explainable property recommendations with a match score and reasons.
11 Provide role-based experiences for buyers and agents through a web interface.	12 Document, verify and test the database design and implementation.

3 Functional Requirements

The requirements below describe what PropMatch must do for each type of user. The last column shows where each one is implemented or evidenced in this report.

ID	Actor	Requirement	Implemented by / evidence
FR-01	Buyer	Log in with email and password; the session is kept in a JWT stored in an httpOnly cookie.	Login page (Fig. 23); users, login_audit
FR-02	Buyer	Browse property listings as cards showing price, locality, bedrooms, bathrooms, area and a sale/rent tag.	Home and browse pages (Figs. 24–28)
FR-03	Buyer	Search and filter by city, locality, bedrooms, property type, budget, amenities and furnishing.	Search section (Fig. 26); idx_prop_search; Mongo SEARCH events
FR-04	Buyer	Open a property to see its images, price, specifications, type, furnishing and listing agent.	Property detail page (Fig. 29)
FR-05	Buyer	Save a property to, and remove it from, a personal favourites list.	Favourites (Fig. 31); favorites table
FR-06	Buyer	Send an enquiry to the agent of an ACTIVE property (message of at least 10 characters).	inquiries; trg_inquiries_before_insert; chk_inq_message
FR-07	Buyer	Book a site visit for a future date and time; overlapping slots on the same property are refused.	site_visits; sp_book_visit; trg_visits_before_insert
FR-08	Buyer	See personalised “For you” recommendations ranked by match %, each with plain-language reasons, and update saved preferences.	For you page (Fig. 30); recommendations_cache
FR-09	Agent	Create, edit, deactivate/reactivate listings and mark them sold or rented.	My listings (Fig. 32); sp_close_property
FR-10	Agent	View a dashboard of total, active and closed listings, enquiries, site visits, average price and enquiries by status.	Dashboard (Fig. 33); v_agent_performance
FR-11	Agent	Manage enquiries and review site-visit requests for own listings.	Dashboard actions (Fig. 33); inquiries, site_visits
FR-12	System	Show only ACTIVE listings of verified agents with active accounts to the public.	v_active_listings
FR-13	System	Record every price change (old price, new price, time) automatically.	trg_properties_price_history; price_history
FR-14	System	Record login events (time, IP address, user agent).	login_audit
FR-15	System	Record buyer activity events: VIEW, SEARCH, FAVORITE, UNFAVORITE, INQUIRY, VISIT_REQUEST.	user_activity (MongoDB)
FR-16	System	Cache each buyer's recommendations for one hour and expire old activity after 180 days.	TTL indexes on MongoDB collections
FR-17	System	Restrict features by role (BUYER, AGENT, ADMIN).	users.role; role-aware navigation

Non-functional requirements

Area	Requirement	How it is met
Data integrity	Invalid or inconsistent data must be rejected.	43 declared constraints (13 PK, 15 FK, 5 UNIQUE, 10 CHECK), ENUM columns, triggers; see Section 9.
Consistency under concurrency	Two buyers must not book the same visit slot.	sp_book_visit locks the property row (FOR UPDATE) inside a transaction.
Security	Passwords and credentials must be protected.	Only password hashes are stored; JWT in an httpOnly cookie; database credentials in environment variables.
Performance	Search and recommendations should respond quickly.	Composite search index, secondary indexes, one-hour recommendation cache.
Data retention	Logs and caches must not grow without limit.	MongoDB TTL indexes (180 days activity, 1 hour cache).

4 Technology Stack and System Architecture

PropMatch pairs a relational core with a document store, and exposes both through a single full-stack web application written in JavaScript on Node.js.

MySQL 8 (relational core)

Stores structured data across **13 tables and 2 views**: users, agent profiles, properties, locations, inquiries, site visits, price history and login audit. Accessed with the **mysql2** driver using raw SQL queries (no ORM).

Frontend: Next.js (App Router) with React

React-based web interface where buyers, renters and agents browse, filter and manage property listings. Pages and components are built with the Next.js App Router inside the same project as the API layer.

Backend: Next.js API routes on Node.js

Server-side route handlers under **app/api** receive application requests and talk to both databases. Authentication uses a **JWT stored in an httpOnly cookie**; database credentials are kept in environment variables.

MongoDB (NoSQL document store)

Runs alongside MySQL as the second database paradigm. It holds the flexible, document-shaped data behind the **property recommendation component**: buyer preferences, activity events and cached recommendations.

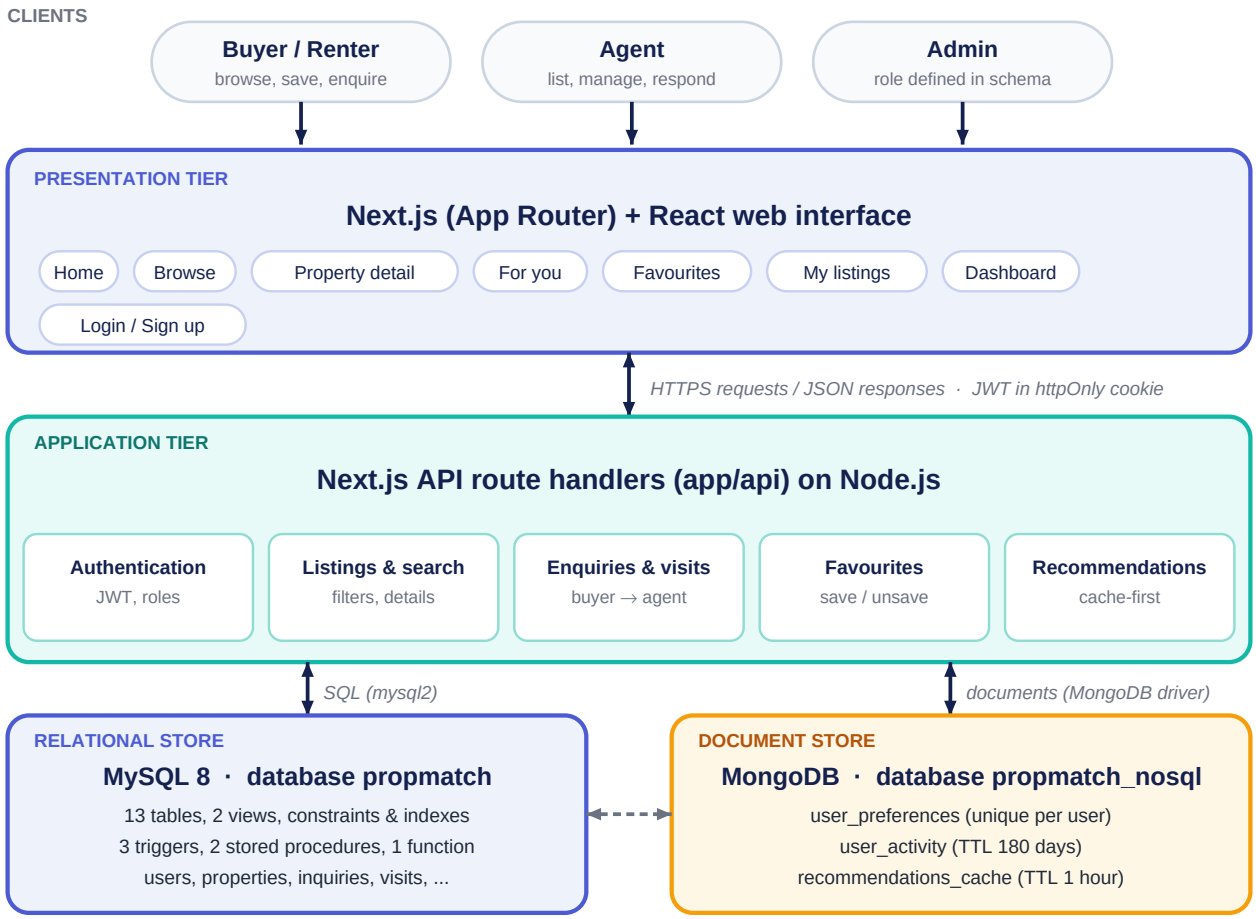
Technology summary

Layer	Technology	Role in PropMatch
Front end	Next.js (App Router), React	Pages and components for buyers, renters and agents.
Back end	Next.js API route handlers on Node.js (JavaScript)	Receives requests, checks the JWT session and role, talks to both databases.
Authentication	JWT in an httpOnly cookie	Session handling; role-aware access (BUYER, AGENT, ADMIN).
Relational database	MySQL 8, mysql2 driver (raw SQL, no ORM)	13 tables, 2 views, triggers, procedures, function, constraints.
NoSQL database	MongoDB (database propmatch_nosql)	Buyer preferences, activity events, cached recommendations.
Configuration	Environment variables	Database credentials are kept out of the source code.

System architecture

PropMatch uses a three-tier architecture. The browser talks only to the Next.js application; the API route handlers are the single place where the two databases are used.

Figure 1 — PropMatch system architecture



The two databases are linked only by identifiers held in the application layer (userId, propertyId); there are no cross-database foreign keys.

Recommendation request flow

Cache-first retrieval (MongoDB + MySQL)

1. The request is authenticated and the caller's role must be BUYER.
2. The buyer is identified from the authenticated userId.
3. recommendations_cache is looked up by userId. If a document exists and its generatedAt timestamp is within one hour, the cached items are returned.
4. Otherwise the recommendations are regenerated from the buyer's stored preferences, recent activity and the active listings, and written back with a new generatedAt.
5. Each returned item carries a numeric score and a list of reasons (for example “Within your budget”), which the interface shows as a match % and tags.

Steps 1–3 follow the NoSQL component report. The MongoDB TTL index removes stale cache documents after about an hour, but because TTL deletion is asynchronous the application still checks freshness itself. The exact scoring formula belongs to the recommender source code and is not reproduced here.

5 Database Choice and Justification

Each data shape and access pattern in PropMatch was matched to the database that handles it best.

Concern	Choice	Justification
Listings, users, agents, locations, enquiries, visits, favourites	**MySQL (relational)**	The data is highly structured and strongly related (a property belongs to one agent, one location and one type; enquiries and visits link buyers to properties). Foreign keys, UNIQUE and CHECK constraints keep it consistent, and joins answer questions such as “listings with their city and agent”.
Booking a visit, closing a listing	**MySQL (ACID transactions)**	sp_book_visit and sp_close_property run inside transactions with row locks, so a slot cannot be double-booked and a closed property, its visits and its enquiries change together or not at all.
Price history and audit trail	**MySQL (triggers, tables)**	A trigger writes every price change to price_history, and login_audit records logins, so history is captured by the database itself regardless of which application path made the change.
Reports and summaries	**MySQL (views, aggregates)**	v_active_listings and v_agent_performance expose joined and aggregated data without duplicating it.
Buyer preferences	**MongoDB (documents)**	Preferences are optional, vary from buyer to buyer and contain lists (cities, localities, type IDs, amenity IDs). A flexible document per buyer avoids a wide table full of NULLs and link tables.
User activity events	**MongoDB (append-only events)**	Events are written constantly, have different metadata depending on type (a source, or a set of search filters) and are only needed for a limited time, so a TTL index expires them after 180 days.
Recommendation results	**MongoDB (cache documents)**	A recommendation list is a nested document with per-item scores and reasons that is read as a whole. It is rebuilt often, so a one-hour TTL index keeps it fresh without manual clean-up.
Semantic / vector search	**Not used**	No embeddings, vector index or nearest-neighbour search exist in the project. Recommendations are rule-based on preferences and activity. Vector search is listed as a future enhancement (Section 18).

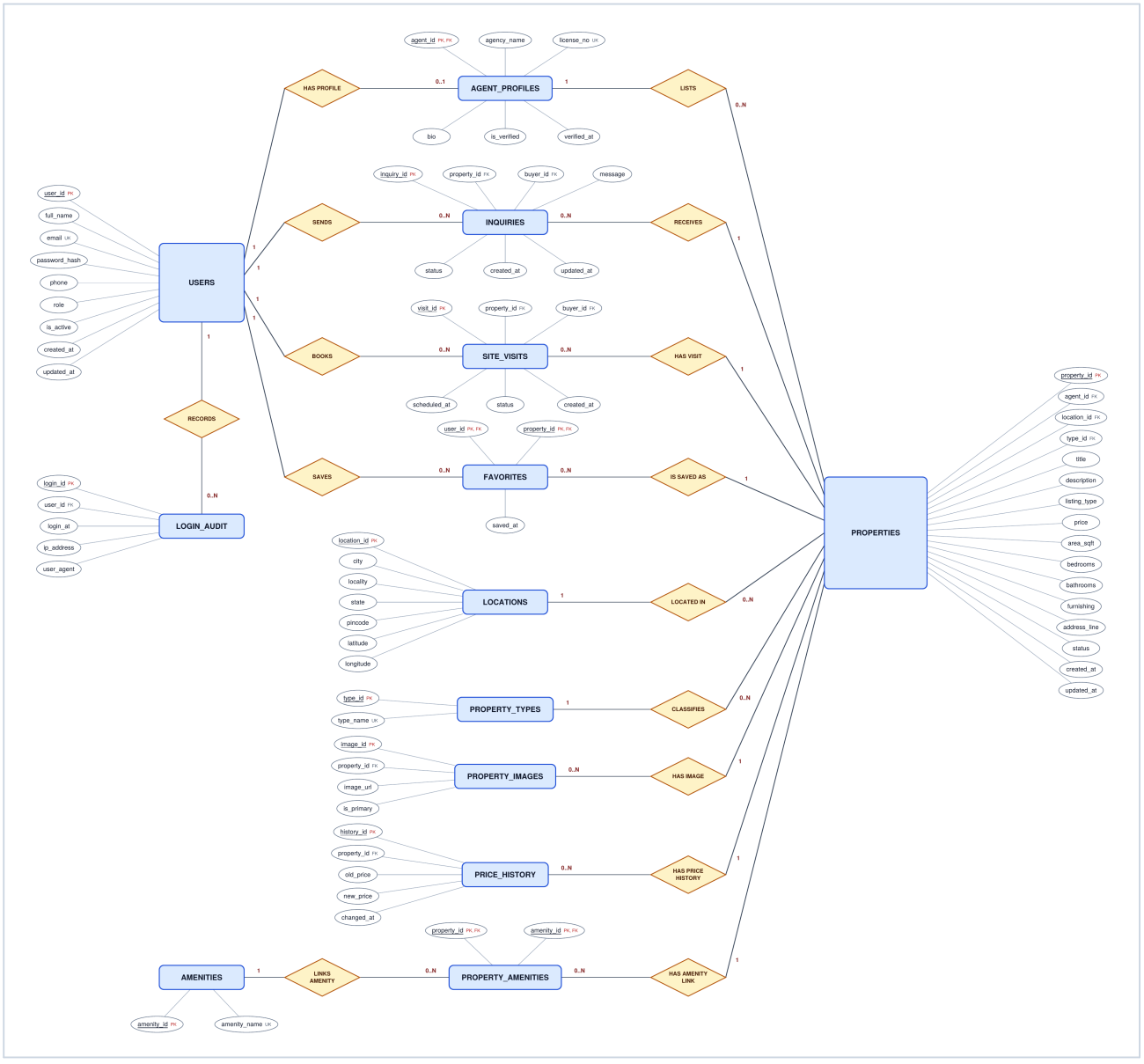
Why not a single database?

Storing everything in MySQL would force flexible, short-lived, per-user data into rigid tables, while storing everything in MongoDB would give up foreign keys, CHECK constraints, triggers and transactions for the core business records. Using both lets each database do what it is best at.

6 Entity-Relationship Design

The ER diagram uses Chen notation: rectangles are entities (tables), diamonds are relationships, ovals are attributes, and the underlined attribute is the primary key. PK, FK and UK tags mark primary, foreign and unique keys. Cardinalities follow the foreign-key constraints: the referenced side is **1** and the referencing side is **0..N**; agent_profiles is a **1 : 0..1** extension of users.

Figure 2 — Chen-notation ER diagram of the propmatch database (13 entities, 15 relationships, 74 attributes)



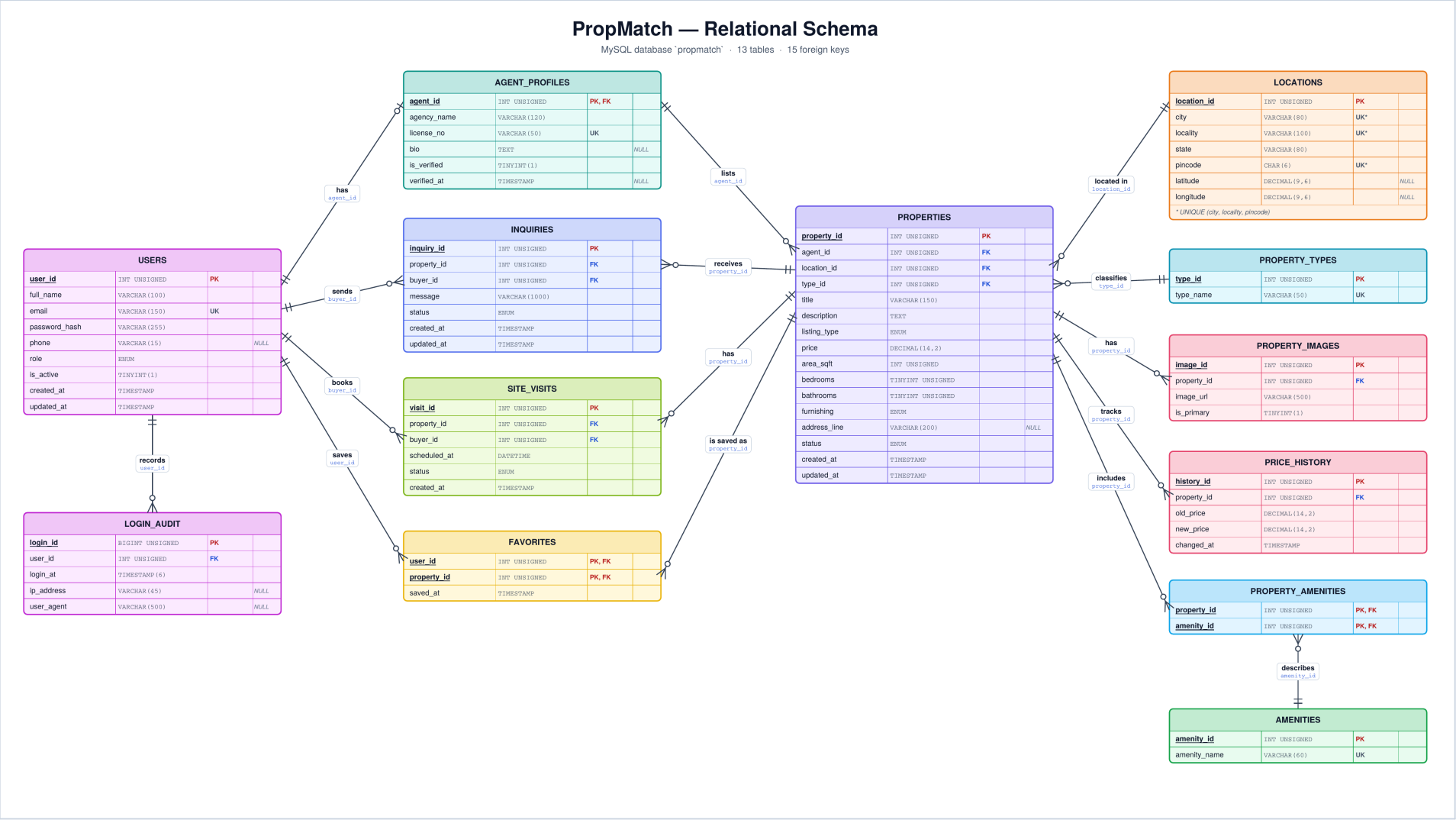
Relationship summary

Relationship	Cardinality	Description
HAS PROFILE	users 1 : 0..1 agent_profiles	A user can have an agent profile (agent_id is both primary key and foreign key).
LISTS	agent_profiles 1 : 0..N properties	An agent can list multiple properties.
LOCATED IN	locations 1 : 0..N properties	A location can be associated with multiple properties.
CLASSIFIES	property_types 1 : 0..N properties	A property type can classify multiple properties.
HAS IMAGE	properties 1 : 0..N property_images	A property can have multiple images.
HAS PRICE HISTORY	properties 1 : 0..N price_history	Price changes are recorded in price history.
SENDS / RECEIVES	users 1 : 0..N inquiries; properties 1 : 0..N inquiries	Buyers send inquiries about properties.
BOOKS / HAS VISIT	users 1 : 0..N site_visits; properties 1 : 0..N site_visits	Buyers request site visits for properties.
SAVES / IS SAVED AS	users 1 : 0..N favorites; properties 1 : 0..N favorites	Users save properties through the favorites table (many-to-many).
HAS AMENITY LINK / LINKS AMENITY	properties 1 : 0..N property_amenities; amenities 1 : 0..N property_amenities	Properties have multiple amenities via property_amenities (many-to-many).
RECORDS	users 1 : 0..N login_audit	User login events are recorded in login audit.

7 Relational Schema

The relational schema shows every table with its columns, data types, keys and nullability. Lines use crow’s-foot notation: a double bar marks the referenced (parent) table, a crow’s foot with a circle marks the referencing (child) table, and each line is labelled with its foreign-key column.

Figure 3 — Relational schema of the propmatch database (13 tables, 15 foreign keys)



Foreign keys

Child column (FK)	Parent column (PK)	On delete	Meaning
agent_profiles.agent_id	users.user_id	CASCADE	A user has at most one agent profile; deleting the user removes it.
login_audit.user_id	users.user_id	CASCADE	Login history is removed with the user.
properties.agent_id	agent_profiles.agent_id	RESTRICT	Every property belongs to an agent.
properties.location_id	locations.location_id	RESTRICT	Every property has a location.
properties.type_id	property_types.type_id	RESTRICT	Every property has a type.
inquiries.property_id	properties.property_id	RESTRICT	Enquiry is about a property.
inquiries.buyer_id	users.user_id	RESTRICT	Enquiry is sent by a user.
site_visits.property_id	properties.property_id	RESTRICT	Visit is for a property.
site_visits.buyer_id	users.user_id	RESTRICT	Visit is requested by a user.
favorites.user_id	users.user_id	RESTRICT	Favourite belongs to a user.
favorites.property_id	properties.property_id	RESTRICT	Favourite points to a property.
property_images.property_id	properties.property_id	RESTRICT	Image belongs to a property.
price_history.property_id	properties.property_id	RESTRICT	Price change belongs to a property.
property_amenities.property_id	properties.property_id	RESTRICT	Link to the property.
property_amenities.amenity_id	amenities.amenity_id	RESTRICT	Link to the amenity.

RESTRICT is MySQL's default when no ON DELETE action is declared. ENUM columns: users.role = BUYER, AGENT, ADMIN; properties.listing_type = SALE, RENT; properties.furnishing = UNFURNISHED, SEMI_FURNISHED, FURNISHED; properties.status = ACTIVE, SOLD, RENTED, INACTIVE; inquiries.status = NEW, CONTACTED, CLOSED; site_visits.status = REQUESTED, CONFIRMED, COMPLETED, CANCELLED.

8 Data Dictionary

The database is named **propmatch** and contains **13 tables and 2 views** (InnoDB, utf8mb4). The table below lists each table's purpose; the pages that follow describe every column.

#	Table	Purpose
1	users	Stores user information, email, password hash, role and account status.
2	agent_profiles	Stores agent-specific details, agency information and verification status.
3	properties	Stores property descriptions, prices, area, listing type, furnishing and availability status.
4	locations	Stores city, locality, state, pincode and optional geographic coordinates.
5	property_types	Stores property categories such as Apartment, Commercial, Independent House, Plot and Villa.
6	amenities	Stores available amenity names.
7	property_images	Stores image references associated with properties.
8	property_amenities	Connects properties with their amenities.
9	favorites	Records properties saved by users.
10	inquiries	Stores messages and inquiry statuses associated with properties and buyers.
11	site_visits	Stores requested or scheduled property visits.
12	price_history	Records previous and updated property prices.
13	login_audit	Records login-related audit information.

MySQL table definitions

1. users

Column	Type	Null	Key	Default	Description
user_id	INT UNSIGNED	NO	PK	AUTO_INCREMENT	Unique user identifier.
full_name	VARCHAR(100)	NO		—	Display name.
email	VARCHAR(150)	NO	UK	—	Login email; unique; CHECK requires an email-like pattern.
password_hash	VARCHAR(255)	NO		—	Hashed password (plain text is never stored).
phone	VARCHAR(15)	YES		NULL	Optional phone; CHECK allows 10–15 digits, +, space or -.
role	ENUM('BUYER','AGENT','ADMIN')	NO		BUYER	Access role.
is_active	TINYINT(1)	NO		1	Account enabled flag.
created_at	TIMESTAMP	NO		CURRENT_TIMESTAMP	Creation time.
updated_at	TIMESTAMP	NO		CURRENT_TIMESTAMP (on update)	Last modification time.

2. agent_profiles

Column	Type	Null	Key	Default	Description
agent_id	INT UNSIGNED	NO	PK, FK	—	Same value as users.user_id; ON DELETE CASCADE.
agency_name	VARCHAR(120)	NO		—	Agency the agent works for.
license_no	VARCHAR(50)	NO	UK	—	Professional licence number; unique.
bio	TEXT	YES		NULL	Free-text biography.
is_verified	TINYINT(1)	NO		0	Verification flag; only verified agents appear in v_active_listings.
verified_at	TIMESTAMP	YES		NULL	When verification happened.

3. properties

Column	Type	Null	Key	Default	Description
property_id	INT UNSIGNED	NO	PK	AUTO_INCREMENT	Unique property identifier.
agent_id	INT UNSIGNED	NO	FK	—	Listing agent → agent_profiles.
location_id	INT UNSIGNED	NO	FK	—	Location → locations.
type_id	INT UNSIGNED	NO	FK	—	Property type → property_types.
title	VARCHAR(150)	NO		—	Listing headline.
description	TEXT	NO		—	Full description; part of the FULLTEXT index.
listing_type	ENUM('SALE','RENT')	NO		—	Whether the property is for sale or rent.
price	DECIMAL(14,2)	NO		—	Sale price or monthly rent; CHECK price > 0.
area_sqft	INT UNSIGNED	NO		—	Built-up area; CHECK area > 0.
bedrooms	TINYINT UNSIGNED	NO		0	Number of bedrooms; CHECK ≤ 50.
bathrooms	TINYINT UNSIGNED	NO		0	Number of bathrooms; CHECK ≤ 20.
furnishing	ENUM('UNFURNISHED','SEMI_FURNISHED','FURNISHED')	NO		UNFURNISHED	Furnishing level.
address_line	VARCHAR(200)	YES		NULL	Optional street address.
status	ENUM('ACTIVE','SOLD','RENTED','INACTIVE')	NO		ACTIVE	Availability state.
created_at	TIMESTAMP	NO		CURRENT_TIMESTAMP	Listing creation time.
updated_at	TIMESTAMP	NO		CURRENT_TIMESTAMP (on update)	Last modification time.

4. locations

Column	Type	Null	Key	Default	Description
location_id	INT UNSIGNED	NO	PK	AUTO_INCREMENT	Unique location identifier.
city	VARCHAR(80)	NO	UK*	—	City name.
locality	VARCHAR(100)	NO	UK*	—	Neighbourhood or locality.
state	VARCHAR(80)	NO		—	State.
pincode	CHAR(6)	NO	UK*	—	Postal code; CHECK requires exactly 6 digits.
latitude	DECIMAL(9,6)	YES		NULL	Optional; CHECK between -90 and 90.
longitude	DECIMAL(9,6)	YES		NULL	Optional; CHECK between -180 and 180.

5. property_types

Column	Type	Null	Key	Default	Description
type_id	INT UNSIGNED	NO	PK	AUTO_INCREMENT	Unique type identifier.
type_name	VARCHAR(50)	NO	UK	—	Category name, for example Apartment or Villa.

6. amenities

Column	Type	Null	Key	Default	Description
amenity_id	INT UNSIGNED	NO	PK	AUTO_INCREMENT	Unique amenity identifier.
amenity_name	VARCHAR(60)	NO	UK	—	Amenity name, for example Parking or Lift.

7. property_images

Column	Type	Null	Key	Default	Description
image_id	INT UNSIGNED	NO	PK	AUTO_INCREMENT	Unique image identifier.
property_id	INT UNSIGNED	NO	FK	—	Owning property.
image_url	VARCHAR(500)	NO		—	Image location (URL).
is_primary	TINYINT(1)	NO		0	Marks the cover image used in listings.

8. property_amenities

Column	Type	Null	Key	Default	Description
property_id	INT UNSIGNED	NO	PK, FK	—	Property that has the amenity.
amenity_id	INT UNSIGNED	NO	PK, FK	—	Amenity offered.

9. favorites

Column	Type	Null	Key	Default	Description
user_id	INT UNSIGNED	NO	PK, FK	—	User who saved the property.
property_id	INT UNSIGNED	NO	PK, FK	—	Saved property.
saved_at	TIMESTAMP	NO		CURRENT_TIMESTAMP	When it was saved.

10. inquiries

Column	Type	Null	Key	Default	Description
inquiry_id	INT UNSIGNED	NO	PK	AUTO_INCREMENT	Unique inquiry identifier.
property_id	INT UNSIGNED	NO	FK	—	Property asked about.
buyer_id	INT UNSIGNED	NO	FK	—	Buyer who sent it → users.
message	VARCHAR(1000)	NO		—	Message text; CHECK at least 10 characters after trimming.
status	ENUM('NEW','CONTACTED','CLOSED')	NO		NEW	Follow-up state.
created_at	TIMESTAMP	NO		CURRENT_TIMESTAMP	Sent time.
updated_at	TIMESTAMP	NO		CURRENT_TIMESTAMP (on update)	Last status change.

11. site_visits

Column	Type	Null	Key	Default	Description
visit_id	INT UNSIGNED	NO	PK	AUTO_INCREMENT	Unique visit identifier.
property_id	INT UNSIGNED	NO	FK	—	Property to visit.
buyer_id	INT UNSIGNED	NO	FK	—	Buyer requesting the visit.
scheduled_at	DATETIME	NO		—	Requested date and time; must be in the future (trigger).
status	ENUM('REQUESTED','CONFIRMED','COMPLETED','CANCELLED')	NO		REQUESTED	Visit state.
created_at	TIMESTAMP	NO		CURRENT_TIMESTAMP	Request time.

12. price_history

Column	Type	Null	Key	Default	Description
history_id	INT UNSIGNED	NO	PK	AUTO_INCREMENT	Unique history identifier.
property_id	INT UNSIGNED	NO	FK	—	Property whose price changed.
old_price	DECIMAL(14,2)	NO		—	Price before the change.
new_price	DECIMAL(14,2)	NO		—	Price after the change.
changed_at	TIMESTAMP	NO		CURRENT_TIMESTAMP	When it changed (written by trigger).

13. login_audit

Column	Type	Null	Key	Default	Description
login_id	BIGINT UNSIGNED	NO	PK	AUTO_INCREMENT	Unique login event identifier.
user_id	INT UNSIGNED	NO	FK	—	User who logged in; ON DELETE CASCADE.
login_at	TIMESTAMP(6)	NO		CURRENT_TIMESTAMP(6)	Login time with microseconds.
ip_address	VARCHAR(45)	YES		NULL	Client IP (long enough for IPv6).
user_agent	VARCHAR(500)	YES		NULL	Browser / client string.

UK* = part of the composite unique key uq_location (city, locality, pincode). PK = primary key, FK = foreign key, UK = unique key.

MongoDB collection definitions (database propmatch_nosql)

user_preferences

Field	Type	Description
_id	ObjectId	Document identifier (generated).
userId	Int	Buyer identifier from MySQL users; unique index.
listingType	String	SALE or RENT.
budgetMin / budgetMax	Number	Preferred price range.
preferredCities	Array of String	Cities of interest.
preferredLocalities	Array of String	Localities of interest.
preferredTypeIds	Array of Int	Property type IDs from MySQL.
minBedrooms	Int	Minimum bedrooms.
furnishing	String or null	Preferred furnishing, if any.
mustHaveAmenityIds	Array of Int	Required amenity IDs from MySQL.
createdAt / updatedAt	Date	Timestamps (both required by the validator).
preferences	Object	Nested form used by the seed data: city, listingType, minPrice, maxPrice, minBedrooms, preferredPropertyTypes, preferredAmenities.

user_activity

Field	Type	Description
_id	ObjectId	Document identifier (generated).
userId	Int	Buyer who acted.
propertyId	Int, Long or null	Property involved; null for searches.
eventType	String (enum)	VIEW, SEARCH, FAVORITE, UNFAVORITE, INQUIRY or VISIT_REQUEST.
metadata	Object (optional)	Event context, such as source or search filters.
createdAt	Date	Event time; TTL index expires it after 180 days.

recommendations_cache

Field	Type	Description
_id	ObjectId	Document identifier (generated).
userId	Int	Buyer the list belongs to; unique index.
items	Array of Object	Ranked recommendations. Each item has propertyId, score and reasons (array of String).
items[].matchScore, title, price, city, locality, ...	Mixed	Display fields stored with each item (observed in the stored documents).
generatedAt	Date	When the list was built; TTL index expires it after 3,600 seconds.

9 Keys, Constraints and Indexes

Constraint	What it does	Example in PropMatch
PRIMARY KEY	Uniquely identifies each record in a table.	property_id identifies a property; favorites uses the composite key (user_id, property_id).
FOREIGN KEY	Maintains relationships between tables.	properties.location_id references locations.location_id.
UNIQUE	Prevents duplicate values in designated columns or combinations.	User email addresses must be unique; (city, locality, pincode) is unique.
NOT NULL	Ensures required fields cannot be left empty.	Mandatory fields such as price and listing type.
CHECK	Restricts values to specified conditions.	Positive prices and areas; valid coordinate ranges; 6-digit pincodes; enquiry message length.
DEFAULT	Supplies a predefined value when none is given.	A new property defaults to ACTIVE status.
ENUM	Restricts a column to predefined values.	Listing type is either SALE or RENT.

Constraint inventory

Querying information_schema.TABLE_CONSTRAINTS on the project database returns **43 constraints**: 13 PRIMARY KEY, 15 FOREIGN KEY, 5 UNIQUE and 10 CHECK (Figure 36). The 10 CHECK constraints are chk_users_email, chk_users_phone, chk_lat, chk_lng, chk_pincode, chk_prop_price, chk_prop_area, chk_prop_bedrooms, chk_prop_baths and chk_inq_message. The 5 UNIQUE constraints are uq_users_email, uq_agent_license, uq_amenity_name, uq_type_name and uq_location.

Indexes

Table	Index	Columns	Purpose
properties	`idx_prop_search`	status, listing_type, location_id, price	Composite index for the main search filter.
properties	`ftx_prop_text`	title, description (FULLTEXT)	Keyword search in listings.
properties	`idx_prop_agent / idx_prop_type`	agent_id / type_id	Agent and type lookups; foreign-key support.
inquiries	`idx_inq_property / idx_inq_buyer`	(property_id, status) / buyer_id	Agent inbox by status; buyer history.
site_visits	`idx_visit_slot / idx_visit_buyer`	(property_id, scheduled_at) / buyer_id	Slot-conflict checks; buyer history.
price_history	`idx_ph_property`	property_id, changed_at	Price timeline per property.
login_audit	`idx_login_audit_user_time / _time`	(user_id, login_at DESC) / login_at DESC	Recent logins per user and overall.
property_images	`idx_img_property`	property_id	Images of a property.
users, agent_profiles, locations, property_types, amenities	`uq_* (unique)`	email; license_no; (city, locality, pincode); type_name; amenity_name	Uniqueness and fast lookup.

MongoDB indexes are described in Section 11.

10 MySQL Implementation

The relational database is created with CREATE TABLE statements (InnoDB engine, utf8mb4 character set), populated with INSERT statements, and extended with triggers, a function, stored procedures and views. The following SQL operations were implemented and demonstrated:

CREATE
Create database objects

INSERT
Add records

SELECT
Retrieve records

UPDATE
Modify records

DELETE
Remove records

WHERE
Filter by a condition

ORDER BY
Sort retrieved records

DISTINCT
Retrieve unique values

GROUP BY
Group for aggregates

COUNT()
Count records

HAVING
Filter grouped results

JOIN
Combine related tables

Programmable objects

Object	Type	What it does
`trg_inquiries_before_insert`	Trigger (BEFORE INSERT, inquiries)	Rejects an inquiry unless the property exists and is ACTIVE (PROPERTY_NOT_AVAILABLE).
`trg_properties_price_history`	Trigger (AFTER UPDATE, properties)	When the price changes, inserts the old and new price into price_history.
`trg_visits_before_insert`	Trigger (BEFORE INSERT, site_visits)	Rejects visits scheduled at or before the current time (VISIT_MUST_BE_IN_FUTURE).
`fn_price_per_sqft`	Function (deterministic)	Returns price ÷ area rounded to 2 decimals; NULLIF avoids division by zero.
`sp_book_visit`	Stored procedure (transaction)	Locks the property, checks it is ACTIVE, refuses a slot within 59 minutes of an existing REQUESTED/CONFIRMED visit, then inserts the visit.
`sp_close_property`	Stored procedure (transaction)	Verifies ownership (or admin), sets status to SOLD or RENTED, cancels open visits and closes open inquiries.
`v_active_listings`	View	Joins properties, types, locations, agents and users; only ACTIVE listings of verified agents with active accounts; adds price_per_sqft and the primary image.
`v_agent_performance`	View	Per-agent totals: listings, active, closed, inquiries, visits and average listing price.

Triggers and function

```
CREATE TRIGGER trg_inquiries_before_insert BEFORE INSERT ON inquiries FOR EACH ROW
BEGIN
    DECLARE v_status VARCHAR(20);
    SELECT status INTO v_status FROM properties WHERE property_id = NEW.property_id;
    IF v_status IS NULL OR v_status <> 'ACTIVE' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'PROPERTY_NOT_AVAILABLE';
    END IF;
END;
```

```
CREATE TRIGGER trg_properties_price_history AFTER UPDATE ON properties FOR EACH ROW
BEGIN
    IF NEW.price <> OLD.price THEN
        INSERT INTO price_history (property_id, old_price, new_price)
        VALUES (OLD.property_id, OLD.price, NEW.price);
    END IF;
END;
```

```
CREATE TRIGGER trg_visits_before_insert BEFORE INSERT ON site_visits FOR EACH ROW
BEGIN
    IF NEW.scheduled_at <= NOW() THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'VISIT_MUST_BE_IN_FUTURE';
    END IF;
END;
```

```
CREATE FUNCTION fn_price_per_sqft(p_price DECIMAL(14,2), p_area INT UNSIGNED)
RETURNS DECIMAL(12,2) DETERMINISTIC
BEGIN
    RETURN ROUND(p_price / NULLIF(p_area, 0), 2);
END;
```

Stored procedures

```
CREATE PROCEDURE sp_book_visit(IN p_property_id INT UNSIGNED, IN p_buyer_id INT UNSIGNED,
                               IN p_scheduled_at DATETIME, OUT p_visit_id INT UNSIGNED)
BEGIN
    DECLARE v_status VARCHAR(20);
    DECLARE v_conflicts INT DEFAULT 0;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;

    START TRANSACTION;
    SELECT status INTO v_status FROM properties WHERE property_id = p_property_id FOR UPDATE;
    IF v_status IS NULL OR v_status <> 'ACTIVE' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'PROPERTY_NOT_AVAILABLE';
    END IF;

    SELECT COUNT(*) INTO v_conflicts FROM site_visits
    WHERE property_id = p_property_id AND status IN ('REQUESTED','CONFIRMED')
      AND scheduled_at BETWEEN p_scheduled_at - INTERVAL 59 MINUTE
                           AND p_scheduled_at + INTERVAL 59 MINUTE;
    IF v_conflicts > 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'SLOT_ALREADY_BOOKED';
    END IF;

    INSERT INTO site_visits (property_id, buyer_id, scheduled_at)
    VALUES (p_property_id, p_buyer_id, p_scheduled_at);
    SET p_visit_id = LAST_INSERT_ID();
    COMMIT;
END;
```

```
CREATE PROCEDURE sp_close_property(IN p_property_id INT UNSIGNED, IN p_agent_id INT UNSIGNED,
                                   IN p_is_admin BOOLEAN)
BEGIN
    DECLARE v_owner INT UNSIGNED;  DECLARE v_listing VARCHAR(10);
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;

    START TRANSACTION;
    SELECT agent_id, listing_type INTO v_owner, v_listing
    FROM properties WHERE property_id = p_property_id FOR UPDATE;
    IF v_owner IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'PROPERTY_NOT_FOUND';
    END IF;
    IF v_owner <> p_agent_id AND p_is_admin = FALSE THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'NOT_PROPERTY_OWNER';
    END IF;

    UPDATE properties SET status = IF(v_listing = 'SALE', 'SOLD', 'RENTED')
    WHERE property_id = p_property_id;
    UPDATE site_visits SET status = 'CANCELLED'
    WHERE property_id = p_property_id AND status IN ('REQUESTED','CONFIRMED');
    UPDATE inquiries SET status = 'CLOSED'
    WHERE property_id = p_property_id AND status <> 'CLOSED';
    COMMIT;
END;
```

Views

```
CREATE VIEW v_active_listings AS
SELECT p.property_id, p.title, p.listing_type, p.price, p.area_sqft, p.bedrooms, p.bathrooms,
       p.furnishing, p.status, p.created_at, fn_price_per_sqft(p.price, p.area_sqft) AS price_per_sqft,
       pt.type_id, pt.type_name, l.location_id, l.city, l.locality, l.state, l.pincode,
       ap.agent_id, u.full_name AS agent_name, ap.agency_name,
       (SELECT i.image_url FROM property_images i
         WHERE i.property_id = p.property_id AND i.is_primary = TRUE LIMIT 1) AS image_url
FROM properties p
JOIN property_types pt ON pt.type_id = p.type_id
JOIN locations l       ON l.location_id = p.location_id
JOIN agent_profiles ap ON ap.agent_id = p.agent_id
JOIN users u           ON u.user_id = ap.agent_id
WHERE p.status = 'ACTIVE' AND ap.is_verified = TRUE AND u.is_active = TRUE;
```

```
CREATE VIEW v_agent_performance AS
SELECT ap.agent_id, u.full_name AS agent_name, ap.agency_name, ap.is_verified,
       COUNT(DISTINCT p.property_id) AS total_listings,
       COUNT(DISTINCT CASE WHEN p.status = 'ACTIVE' THEN p.property_id END) AS active_listings,
       COUNT(DISTINCT CASE WHEN p.status IN ('SOLD','RENTED') THEN p.property_id END) AS closed_listings,
       COUNT(DISTINCT i.inquiry_id) AS total_inquiries, COUNT(DISTINCT v.visit_id) AS total_visits,
       ROUND(AVG(p.price), 2) AS avg_listing_price
FROM agent_profiles ap
JOIN users u ON u.user_id = ap.agent_id
LEFT JOIN properties p  ON p.agent_id = ap.agent_id
LEFT JOIN inquiries i   ON i.property_id = p.property_id
LEFT JOIN site_visits v ON v.property_id = p.property_id
GROUP BY ap.agent_id, u.full_name, ap.agency_name, ap.is_verified;
```

Data loaded in the project database

The project database holds **13 users, 32 properties, 8 locations, 17 inquiries, 31 property images, 8 favorites and 18 site visits** (record counts in Figure 37).

11 NoSQL / Vector Database Implementation (MongoDB)

11.1 Role of MongoDB

MongoDB runs alongside MySQL in the database **propmatch_nosql**. It stores flexible buyer-preference records, user-activity events and cached property recommendations. MySQL remains the source of truth for users and properties; MongoDB documents refer to them only through the identifiers **userId** and **propertyId**, and the application keeps these consistent. The MongoDB setup script defines no relational foreign keys.

Collection	Purpose	Documents in project
`user_preferences`	A buyer's property-search preferences (one document per user).	2
`user_activity`	Buyer interaction events: views, searches, favourites, enquiries and visit requests.	201
`recommendations_cache`	Generated recommendation results for a buyer (one document per user).	1

11.2 Collections and sample documents

user_preferences. The validator requires userId, listingType, budgetMin, budgetMax, preferredCities, preferredLocalities, preferredTypeIds, minBedrooms, furnishing, mustHaveAmenityIds, createdAt and updatedAt. The intended document shape is:

```
{
  "userId": NumberInt(5),            "listingType": "RENT",
  "budgetMin": 30000,                "budgetMax": 60000,
  "preferredCities": ["Bengaluru"],  "preferredLocalities": [],
  "preferredTypeIds": [],            "minBedrooms": 2,
  "furnishing": null,                "mustHaveAmenityIds": [],
  "createdAt": ISODate("..."),       "updatedAt": ISODate("...")
}
```

user_activity. The validator requires userId, eventType and createdAt; propertyId may be an integer, a long or null; eventType must be one of VIEW, SEARCH, FAVORITE, UNFAVORITE, INQUIRY or VISIT_REQUEST; metadata is an optional object.

```
{ "userId": NumberInt(5), "propertyId": NumberInt(6), "eventType": "VIEW",
  "metadata": { "source": "recommendations" }, "createdAt": ISODate("...") }
```

recommendations_cache. The validator requires userId, items and generatedAt; every item must contain propertyId, score and reasons (an array of strings).

```
{ "userId": NumberInt(5),
  "items": [ { "propertyId": NumberInt(6), "score": 0.95,
               "reasons": ["Matches preferred city, rent range and bedrooms"] } ],
  "generatedAt": ISODate("...") }
```

The three screenshots below show the documents as they are stored in the project database.

Figure 4 — user_activity — findOne() and find().limit(5)

Each event holds userId, propertyId, eventType, optional metadata and a createdAt timestamp.

```
propmatch_nosql> db.user_activity.findOne()
{
  _id: ObjectId('6abd216d6b9218e1c7aea7bf'),
  userId: 5,
  propertyId: 6,
  eventType: 'VIEW',
  metadata: { source: 'recommendations' },
  createdAt: ISODate('2026-09-30T14:49:17.693Z')
}
propmatch_nosql> db.user_activity.find().limit(5)
[
  {
    _id: ObjectId('6abd216d6b9218e1c7aea7bf'),
    userId: 5,
    propertyId: 6,
    eventType: 'VIEW',
    metadata: { source: 'recommendations' },
    createdAt: ISODate('2026-09-30T14:49:17.693Z')
  },
  {
    _id: ObjectId('6abd216d6b9218e1c7aea7c0'),
    userId: 5,
    propertyId: 29,
    eventType: 'VIEW',
    metadata: { source: 'search' },
    createdAt: ISODate('2026-09-30T14:49:17.693Z')
  },
  {
    _id: ObjectId('6abd216d6b9218e1c7aea7c1'),
    userId: 5,
    propertyId: 6,
    eventType: 'FAVORITE',
    metadata: {},
    createdAt: ISODate('2026-09-30T14:49:17.693Z')
  },
  {
    _id: ObjectId('6abd216d6b9218e1c7aea7c2'),
    userId: 5,
    propertyId: null,
    eventType: 'SEARCH',
    metadata: {
      filters: {
        city: 'Bengaluru',
        listingType: 'RENT',
        minPrice: 30000,
        maxPrice: 60000
      }
    },
    createdAt: ISODate('2026-09-30T14:49:17.693Z')
  },
  {
    _id: ObjectId('6abd216d6b9218e1c7aea7c3'),
    userId: 6,
    propertyId: 1,
    eventType: 'VIEW',
    metadata: { source: 'search' },
    createdAt: ISODate('2026-09-30T14:49:17.693Z')
  }
]
propmatch_nosql>
```

Figure 5 — user_preferences — stored buyer preference documents

User 5 has the flat validator fields plus the nested preferences object; user 6 has the nested seed form.

```
propmatch_nosql> db.user_preferences.find().limit(5)
[
  {
    _id: ObjectId('6abd216d6b9218e1c7aea7bd'),
    userId: 5,
    preferences: {
      city: 'Bengaluru',
      listingType: 'RENT',
      minPrice: 30000,
      maxPrice: 60000,
      minBedrooms: 2,
      preferredPropertyTypes: [ 'Apartment' ],
      preferredAmenities: [ 'Parking', 'Lift' ]
    },
    updatedAt: ISODate('2026-10-02T15:44:15.304Z'),
    budgetMax: 60000,
    budgetMin: 30000,
    furnishing: null,
    listingType: 'RENT',
    minBedrooms: 2,
    mustHaveAmenityIds: [ 3, 6 ],
    preferredCities: [ 'Bengaluru' ],
    preferredLocalities: [ 'HSR Layout', 'JP Nagar' ],
    preferredTypeIds: [ 1 ]
  },
  {
    _id: ObjectId('6abd216d6b9218e1c7aea7be'),
    userId: 6,
    preferences: {
      city: 'Bengaluru',
      listingType: 'SALE',
      minPrice: 5000000,
      maxPrice: 15000000,
      minBedrooms: 2,
      preferredPropertyTypes: [ 'Apartment', 'Villa' ],
      preferredAmenities: [ 'Parking', 'Security' ]
    },
    updatedAt: ISODate('2026-09-30T14:49:17.692Z')
  }
]
```

Figure 6 — recommendations_cache — cached recommendation document (part 1)

The document holds userId, generatedAt and an items array; each item shows its matchScore, score and reasons.

```
propmatch_nosql> db.recommendations_cache.find().limit(5)
[
  {
    _id: ObjectId('6ae59f3aaca4bee8a4ce06ce'),
    userId: 5,
    generatedAt: ISODate('2026-10-31T07:56:42.206Z'),
    items: [
      {
        propertyId: 6,
        title: 'HSR Rental Apartment',
        description: null,
        listingType: 'RENT',
        price: 42000,
        areaSqft: 1250,
        bedrooms: 2,
        bathrooms: 2,
        furnishing: 'FURNISHED',
        typeId: 1,
        typeName: 'Apartment',
        locationId: 2,
        city: 'Bengaluru',
        locality: 'HSR Layout',
        primaryImage: 'https://picsum.photos/seed/propmatch6/800/600',
        pricePerSqft: null,
        matchScore: 99.1,
        score: 99.1,
        reasons: [
          'Within your budget',
          'Preferred locality',
          'Preferred property type',
          'Meets your bedroom needs',
          'Has your must-have amenities',
          'Similar to homes you viewed'
        ]
      },
      {
        propertyId: 29,
        title: 'JP Nagar Rental Apartment',
        description: null,
        listingType: 'RENT',
        price: 45000,
        areaSqft: 1250,
        bedrooms: 2,
        bathrooms: 2,
        furnishing: 'FURNISHED',
        typeId: 1,
        typeName: 'Apartment',
        locationId: 8,
        city: 'Bengaluru',
        locality: 'JP Nagar',
        primaryImage: 'https://picsum.photos/seed/propmatch29/800/600',
        pricePerSqft: null,
        matchScore: 87.6,
        score: 87.6,
        reasons: [
          'Within your budget',
          'Preferred locality',
          'Preferred property type',
```

Figure 7 — recommendations_cache — cached recommendation document (part 2)

Continuation of the same find().limit(5) output: remaining items with their reasons.

```
        score: 87.6,
        reasons: [
          'Within your budget',
          'Preferred locality',
          'Preferred property type',
          'Meets your bedroom needs',
          'Similar to homes you viewed'
        ]
      },
      {
        propertyId: 13,
        title: 'Koramangala Rental Home',
        description: null,
        listingType: 'RENT',
        price: 65000,
        areaSqft: 1300,
        bedrooms: 2,
        bathrooms: 2,
        furnishing: 'FURNISHED',
        typeId: 1,
        typeName: 'Apartment',
        locationId: 4,
        city: 'Bengaluru',
        locality: 'Koramangala',
        primaryImage: 'https://picsum.photos/seed/propmatch13/800/600',
        pricePerSqft: null,
        matchScore: 65,
        score: 65,
        reasons: [
          'Within your budget',
          'Preferred city',
          'Preferred property type',
          'Meets your bedroom needs'
        ]
      },
      {
        propertyId: 14,
        title: 'Koramangala Office Space',
        description: null,
        listingType: 'RENT',
        price: 95000,
        areaSqft: 2200,
        bedrooms: 0,
        bathrooms: 2,
        furnishing: 'SEMI_FURNISHED',
        typeId: 2,
        typeName: 'Independent House',
        locationId: 4,
        city: 'Bengaluru',
        locality: 'Koramangala',
        primaryImage: 'https://picsum.photos/seed/propmatch14/800/600',
        pricePerSqft: null,
        matchScore: 15,
        score: 15,
        reasons: [ 'Preferred city' ]
      }
    ]
  }
]
propmatch_nosql>
```

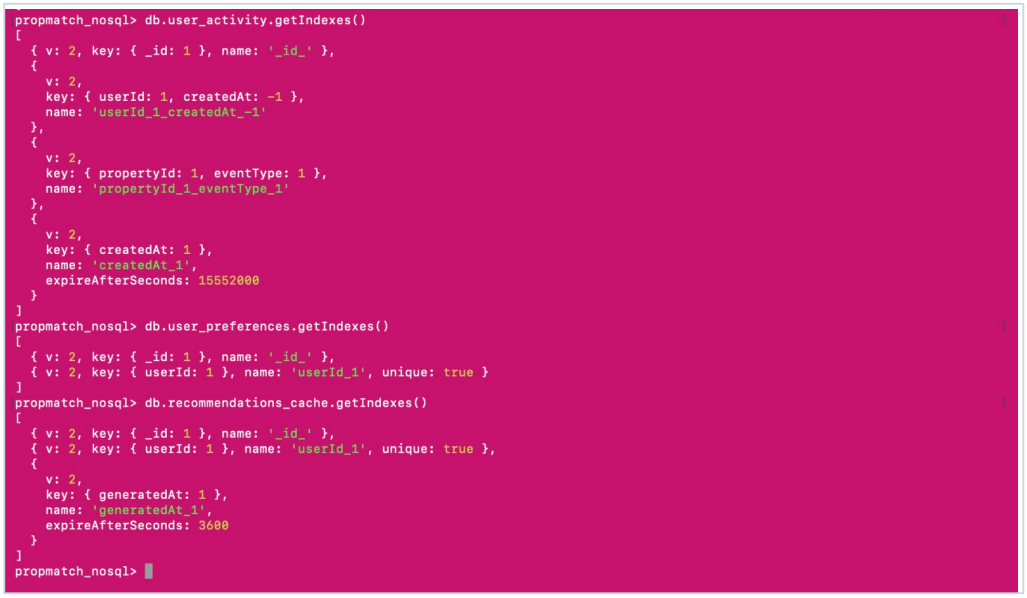
11.3 Indexes and their purpose

Collection	Index definition	Purpose
user_preferences	{ userId: 1 }, unique	Fast buyer lookup; prevents more than one preference document per user.
user_activity	{ userId: 1, createdAt: -1 }	Retrieves a buyer's activity, newest first.
user_activity	{ propertyId: 1, eventType: 1 }	Supports filtering events by property and event type.
user_activity	{ createdAt: 1 }, TTL 15552000	Expires activity records after 15,552,000 seconds (180 days).
recommendations_cache	{ userId: 1 }, unique	Fast cache lookup; enforces one cache document per user.
recommendations_cache	{ generatedAt: 1 }, TTL 3600	Expires cache documents about one hour after generatedAt.

TTL deletion is performed asynchronously by MongoDB, so application code still checks whether a cache entry is fresh before using it.

Figure 8 — getIndexes() on user_activity, user_preferences and recommendations_cache

Confirms the compound, unique and TTL indexes (expireAfterSeconds 15552000 and 3600).



11.4 Data validation

The setup script attaches a JSON Schema validator to all three collections with **validationLevel = moderate** and **validationAction = warn**. Documents that do not match the validator therefore produce warnings instead of being rejected, which is why the stored documents should follow the validator shape.

Observations on the stored data: the cached recommendation document uses the validator's items array with a reasons array on each item. The preference document for user 5 carries the flat validator fields (budgetMin, budgetMax, preferredCities, preferredTypeIds, mustHaveAmenityIds, ...) alongside the nested preferences object, but no createdAt field is visible in the output; the preference document for user 6 still has only the nested seed form. Aligning all preference documents with the validator (including createdAt) is recorded as a follow-up in Section 18.

11.5 Embeddings and vector search

Status: not implemented

The MongoDB setup and seed scripts define no embedding fields, vector indexes, vector-search index or approximate-nearest-neighbour retrieval. The recommendation flow is based on buyer preferences and activity plus a cached result. PropMatch is therefore a MongoDB-backed preference/activity recommendation system, not a vector-search system. If vector search is added later, the report should state the embedding model, vector dimensions, the text or features embedded, the storage field, the index configuration, the similarity metric and the query implementation.

11.6 Metadata and retrieval strategy

Metadata field	Meaning
`userId`	Buyer associated with the record; the key shared by preferences, activity and recommendations.
`propertyId`	Property associated with an event or recommendation (links to MySQL properties).
`eventType`	Kind of interaction: VIEW, SEARCH, FAVORITE, UNFAVORITE, INQUIRY, VISIT_REQUEST.
`metadata`	Event context, for example source = "recommendations" or "search", or nested search filters (city, listingType, price range, furnishing).
`createdAt / generatedAt / updatedAt`	Timestamps used for ordering, freshness checks and TTL expiry.
`score / matchScore / reasons`	Numeric ranking value and plain-language explanations stored for every recommended property.

Retrieval follows the cache-first flow described in Section 4: the cached document is returned while it is fresh (under one hour), otherwise it is rebuilt from the buyer's preferences and activity and stored again. The user_preferences and user_activity collections supply the flexible buyer profile used for ranking.

12 Important Queries

12.1 MySQL queries

The queries below were run in the MySQL command-line client against the propmatch database. Each heading states what the query demonstrates.

Figure 9 — SELECT — first five property records

SELECT * FROM properties LIMIT 5;

```
mysql> SELECT * FROM properties LIMIT 5;
+-------------+-----------+-------------+---------+-----------------------------+-----------------------------
```

Figure 11 — ORDER BY — all properties sorted by price (ascending)

Sorts all 32 properties from 38,000 rentals up to 28,000,000 villas.

```
mysql> SELECT title, price
    -> FROM properties
    -> ORDER BY price ASC;
+-----------------------------------+--------------+
| title                             | price        |
+-----------------------------------+--------------+
| Marathahalli Rental Flat          |     38000.00 |
| HSR Rental Apartment              |     42000.00 |
| JP Nagar Rental Apartment         |     45000.00 |
| Koramangala Rental Home           |     65000.00 |
| Koramangala Office Space          |     95000.00 |
| Electronic City 2 BHK             |   6200000.00 |
| Marathahalli Family Apartment     |   6800000.00 |
| Yelahanka New Apartment           |   7200000.00 |
| Affordable HSR Apartment          |   7400000.00 |
| JP Nagar 2 BHK                    |   7500000.00 |
| Modern Apartment in Whitefield    |   8200000.00 |
| Electronic City 3 BHK             |   8500000.00 |
| Marathahalli 3 BHK                |   9200000.00 |
| API Test Property                 |   9500000.00 |
| Agent One Updated Apartment       |   9500000.00 |
| Indiranagar 2 BHK                 |   9800000.00 |
| Electronic City Commercial Space  |  11000000.00 |
| Premium 3 BHK Whitefield Home     |  11500000.00 |
| JP Nagar Premium Apartment        |  11800000.00 |
| Yelahanka Independent House       |  12000000.00 |
| Electronic City Villa             |  13500000.00 |
| HSR Independent House             |  13500000.00 |
| Indiranagar Premium Flat          |  14500000.00 |
| Koramangala Luxury Apartment      |  15500000.00 |
| JP Nagar Independent House        |  15500000.00 |
| Marathahalli Villa                |  16000000.00 |
| Indiranagar Commercial Property   |  18000000.00 |
| Yelahanka Luxury Villa            |  18500000.00 |
| JP Nagar Commercial Building      |  19500000.00 |
| Indiranagar Villa                 |  22500000.00 |
| Luxury Villa Whitefield           |  25000000.00 |
| Koramangala Independent Villa     |  28000000.00 |
+-----------------------------------+--------------+
32 rows in set (0.000 sec)
```

Figure 12 — DISTINCT — unique property statuses

Lists ACTIVE, SOLD, RENTED and INACTIVE.

```
mysql> SELECT DISTINCT status
    -> FROM properties;
+----------+
| status   |
+----------+
| ACTIVE   |
| SOLD     |
| RENTED   |
| INACTIVE |
+----------+
4 rows in set (0.001 sec)
```

Figure 13 — GROUP BY + COUNT() — properties per status

27 ACTIVE, 2 SOLD, 1 RENTED, 2 INACTIVE.

```
mysql> SELECT status, COUNT(*) AS total_properties
    -> FROM properties
    -> GROUP BY status;
+----------+------------------+
| status   | total_properties |
+----------+------------------+
| ACTIVE   |               27 |
| SOLD     |                2 |
| RENTED   |                1 |
| INACTIVE |                2 |
+----------+------------------+
4 rows in set (0.000 sec)
```

Figure 14 — HAVING — statuses with more than one property

RENTED (1) is filtered out.

```
mysql> SELECT status, COUNT(*) AS total_properties
    -> FROM properties
    -> GROUP BY status
    -> HAVING COUNT(*) > 1;
+----------+------------------+
| status   | total_properties |
+----------+------------------+
| ACTIVE   |               27 |
| SOLD     |                2 |
| INACTIVE |                2 |
+----------+------------------+
3 rows in set (0.000 sec)
```

Figure 15 — JOIN — properties with their city and locality

Joins properties with locations on location_id; returns 32 rows.

```
mysql> SELECT p.title, p.price, l.city, l.locality
    -> FROM properties p
    -> JOIN locations l
    -> ON p.location_id = l.location_id;
+----------------------------------+-------------+-----------+-----------------+
| title                            | price       | city      | locality        |
+----------------------------------+-------------+-----------+-----------------+
| Electronic City 2 BHK            |  6200000.00 | Bengaluru | Electronic City |
| Electronic City 3 BHK            |  8500000.00 | Bengaluru | Electronic City |
| Electronic City Villa            | 13500000.00 | Bengaluru | Electronic City |
| Electronic City Commercial Space | 11000000.00 | Bengaluru | Electronic City |
| Affordable HSR Apartment         |  7400000.00 | Bengaluru | HSR Layout      |
| HSR Independent House            | 13500000.00 | Bengaluru | HSR Layout      |
| HSR Rental Apartment             |    42000.00 | Bengaluru | HSR Layout      |
| Indiranagar Premium Flat         | 14500000.00 | Bengaluru | Indiranagar     |
| Indiranagar 2 BHK                |  9800000.00 | Bengaluru | Indiranagar     |
| Indiranagar Villa                | 22500000.00 | Bengaluru | Indiranagar     |
| Indiranagar Commercial Property  | 18000000.00 | Bengaluru | Indiranagar     |
| JP Nagar 2 BHK                   |  7500000.00 | Bengaluru | JP Nagar        |
| JP Nagar Premium Apartment       | 11800000.00 | Bengaluru | JP Nagar        |
| JP Nagar Independent House       | 15500000.00 | Bengaluru | JP Nagar        |
| JP Nagar Rental Apartment        |    45000.00 | Bengaluru | JP Nagar        |
| JP Nagar Commercial Building     | 19500000.00 | Bengaluru | JP Nagar        |
| Koramangala Luxury Apartment     | 15500000.00 | Bengaluru | Koramangala     |
| Koramangala Independent Villa    | 28000000.00 | Bengaluru | Koramangala     |
| Koramangala Rental Home          |    65000.00 | Bengaluru | Koramangala     |
| Koramangala Office Space         |    95000.00 | Bengaluru | Koramangala     |
| Marathahalli Family Apartment    |  6800000.00 | Bengaluru | Marathahalli    |
| Marathahalli 3 BHK               |  9200000.00 | Bengaluru | Marathahalli    |
| Marathahalli Villa               | 16000000.00 | Bengaluru | Marathahalli    |
| Marathahalli Rental Flat         |    38000.00 | Bengaluru | Marathahalli    |
| Modern Apartment in Whitefield   |  8200000.00 | Bengaluru | Whitefield      |
| Premium 3 BHK Whitefield Home    | 11500000.00 | Bengaluru | Whitefield      |
| Luxury Villa Whitefield          | 25000000.00 | Bengaluru | Whitefield      |
| Agent One Updated Apartment      |  9500000.00 | Bengaluru | Whitefield      |
| API Test Property                |  9500000.00 | Bengaluru | Whitefield      |
| Yelahanka New Apartment          |  7200000.00 | Bengaluru | Yelahanka       |
| Yelahanka Luxury Villa           | 18500000.00 | Bengaluru | Yelahanka       |
| Yelahanka Independent House      | 12000000.00 | Bengaluru | Yelahanka       |
+----------------------------------+-------------+-----------+-----------------+
32 rows in set (0.001 sec)
```

Figure 16 — View v_active_listings — SELECT * FROM v_active_listings LIMIT 10

Joined property, location, type and agent columns with the calculated price_per_sqft.

```
mysql> SELECT * FROM v_active_listings LIMIT 10;
+-----------+-----+-------------------------------+-------------+----------+-----------+-----------+-
```

Reference queries for the main application features

These parameterised queries show how the application features map to SQL. All of them were executed successfully against a rebuilt copy of the schema (Section 14).

```
-- 1. Search active rentals in a budget (uses idx_prop_search through the view's base table)
SELECT property_id, title, price, price_per_sqft
FROM v_active_listings
WHERE listing_type = 'RENT' AND city = 'Bengaluru' AND price BETWEEN 30000 AND 60000 AND bedrooms >= 2
ORDER BY price;

-- 2. Keyword search in title and description (FULLTEXT index ftx_prop_text)
SELECT property_id, title FROM properties
WHERE MATCH(title, description) AGAINST('villa garden' IN NATURAL LANGUAGE MODE);

-- 3. Properties that have ALL of the chosen amenities
SELECT p.property_id, p.title
FROM properties p JOIN property_amenities pa ON pa.property_id = p.property_id
WHERE pa.amenity_id IN (1, 2)
GROUP BY p.property_id, p.title HAVING COUNT(DISTINCT pa.amenity_id) = 2;

-- 4. Price timeline of one property
SELECT old_price, new_price, changed_at FROM price_history
WHERE property_id = ? ORDER BY changed_at DESC;

-- 5. Agent inbox: enquiries on an agent's listings
SELECT i.inquiry_id, p.title, u.full_name AS buyer, i.message, i.status
FROM inquiries i JOIN properties p ON p.property_id = i.property_id
                 JOIN users u ON u.user_id = i.buyer_id
WHERE p.agent_id = ? ORDER BY i.created_at DESC;

-- 6. A buyer's saved homes
SELECT p.title, f.saved_at FROM favorites f JOIN properties p ON p.property_id = f.property_id
WHERE f.user_id = ? ORDER BY f.saved_at DESC;

-- 7. Average price per sq ft by locality
SELECT city, locality, COUNT(*) AS listings, ROUND(AVG(price_per_sqft), 2) AS avg_psf
FROM v_active_listings GROUP BY city, locality ORDER BY avg_psf DESC;

-- 8. Book a visit / close a listing (transactional procedures)
CALL sp_book_visit(1, 3, '2026-11-10 11:00:00', @visit_id);
CALL sp_close_property(1, 1, FALSE);
```

12.2 MongoDB queries

The queries below were run in the MongoDB shell against propmatch_nosql.

Figure 18 — countDocuments() — documents per collection

201 activity events, 2 preference documents and 1 cached recommendation document.

```
propmatch_nosql> db.user_activity.countDocuments()
201
propmatch_nosql> db.user_preferences.countDocuments()
2
propmatch_nosql> db.recommendations_cache.countDocuments()
1
propmatch_nosql>
```

Figure 19 — distinct("eventType") — event types recorded

All six event types allowed by the validator are present.

```
propmatch_nosql> db.user_activity.distinct("eventType")
[
  'FAVORITE',
  'INQUIRY',
  'SEARCH',
  'UNFAVORITE',
  'VIEW',
  'VISIT_REQUEST'
]
propmatch_nosql>
```

Figure 20 — aggregate() — total activities and activities per event type

$group by null gives 201; $group by eventType and $sort by total gives VIEW 77, SEARCH 56, UNFAVORITE 29, FAVORITE 26, VISIT_REQUEST 8, INQUIRY 5.

```
propmatch_nosql> db.user_activity.aggregate([
|     {
|         $group: {
|             _id: null,
|             totalActivities: { $sum: 1 }
|         }
|     }
| ])
[ { _id: null, totalActivities: 201 } ]
propmatch_nosql> db.user_activity.aggregate([
|     {
|         $group: {
|             _id: "$eventType",
|             total: { $sum: 1 }
|         }
|     },
|     {
|         $sort: { total: -1 }
|     }
| ])
[
  { _id: 'VIEW', total: 77 },
  { _id: 'SEARCH', total: 56 },
  { _id: 'UNFAVORITE', total: 29 },
  { _id: 'FAVORITE', total: 26 },
  { _id: 'VISIT_REQUEST', total: 8 },
  { _id: 'INQUIRY', total: 5 }
]
propmatch_nosql>
```

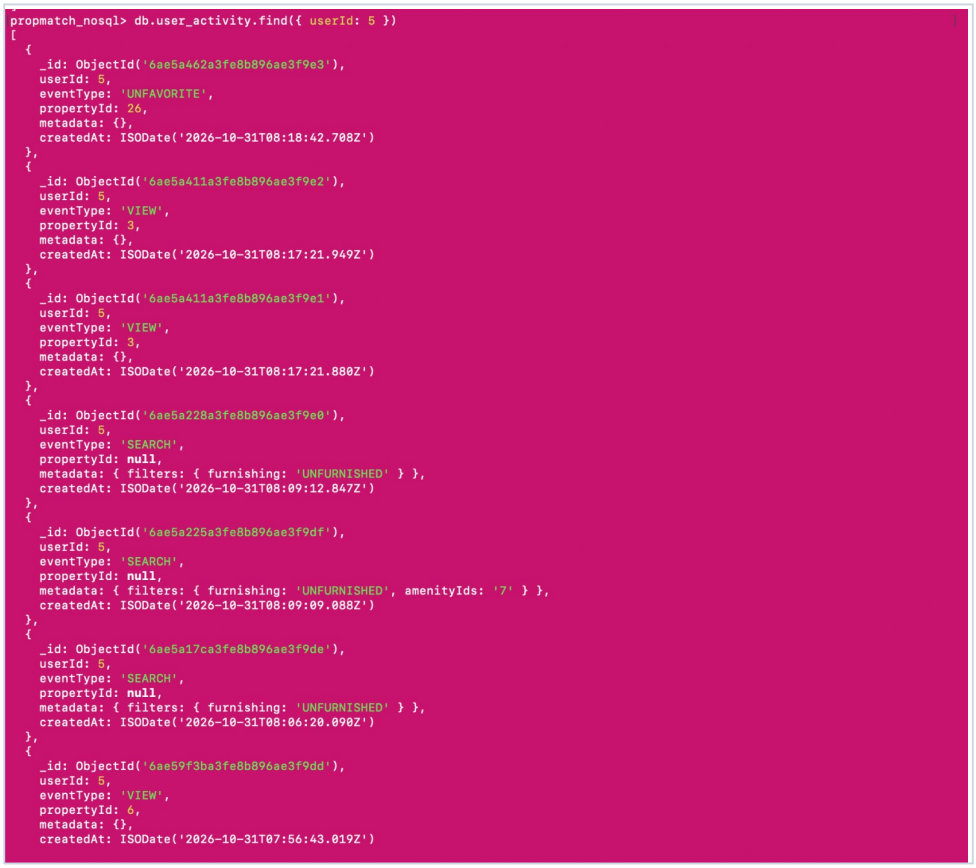
Figure 21 — find({ eventType: "VIEW" }).limit(5) — filtering events by type

Returns view events with their source metadata.

```
propmatch_nosql> db.user_activity.find({ eventType: "VIEW" }).limit(5)
[
  {
    _id: ObjectId('6abd216d6b9218e1c7aea7bf'),
    userId: 5,
    propertyId: 6,
    eventType: 'VIEW',
    metadata: { source: 'recommendations' },
    createdAt: ISODate('2026-09-30T14:49:17.693Z')
  },
  {
    _id: ObjectId('6abd216d6b9218e1c7aea7c0'),
    userId: 5,
    propertyId: 29,
    eventType: 'VIEW',
    metadata: { source: 'search' },
    createdAt: ISODate('2026-09-30T14:49:17.693Z')
  },
  {
    _id: ObjectId('6abd216d6b9218e1c7aea7c3'),
    userId: 6,
    propertyId: 1,
    eventType: 'VIEW',
    metadata: { source: 'search' },
    createdAt: ISODate('2026-09-30T14:49:17.693Z')
  },
  {
    _id: ObjectId('6abfd4b09a6033c010935fae'),
    userId: 5,
    eventType: 'VIEW',
    propertyId: 4,
    metadata: {},
    createdAt: ISODate('2026-10-02T15:58:40.584Z')
  },
  {
    _id: ObjectId('6abfd4b09a6033c010935faf'),
    userId: 5,
    eventType: 'VIEW',
    propertyId: 4,
    metadata: {},
    createdAt: ISODate('2026-10-02T15:58:40.694Z')
  }
]
propmatch_nosql>
```

Figure 22 — find({ userId: 5 }) — one buyer's activity history

Shows UNFAVORITE, VIEW and SEARCH events; search events keep their filters in metadata.



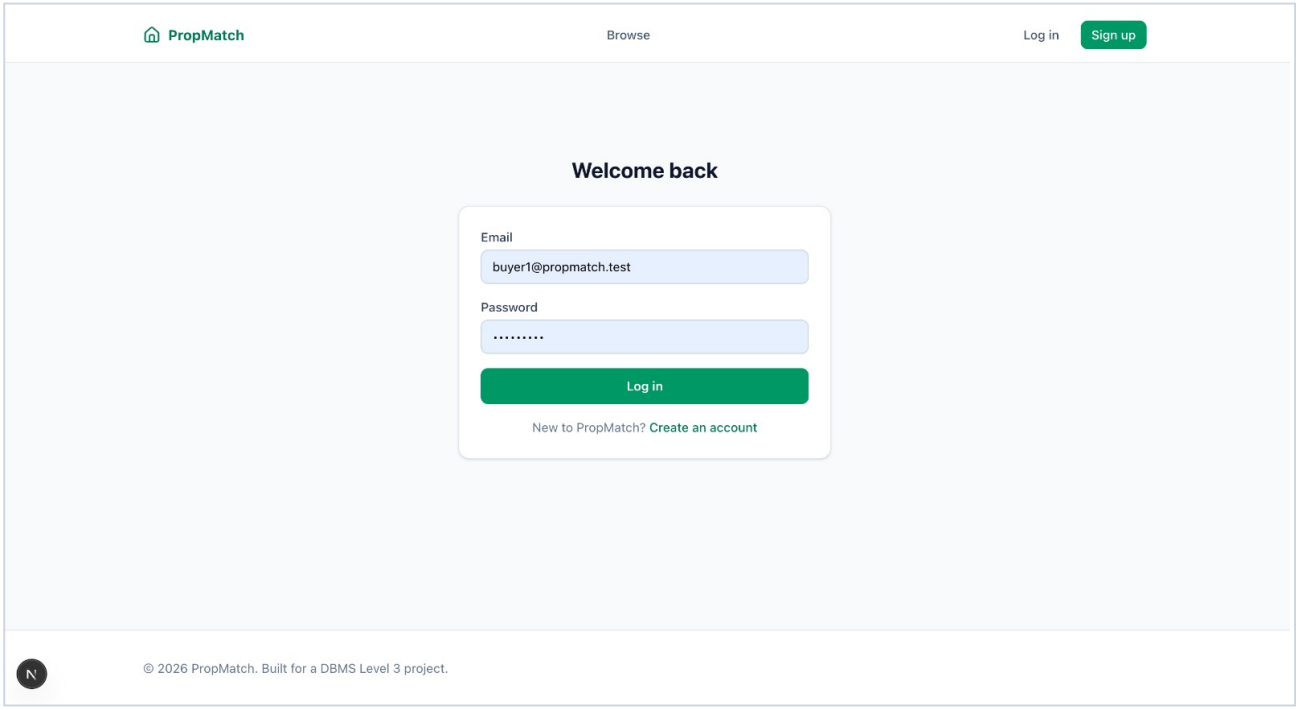
```
propmatch_nosql> db.user_activity.find({ userId: 5 })
[
  {
    _id: ObjectId('6ae5a462a3fe8b896ae3f9e3'),
    userId: 5,
    eventType: 'UNFAVORITE',
    propertyId: 26,
    metadata: {},
    createdAt: ISODate('2026-10-31T08:18:42.708Z')
  },
  {
    _id: ObjectId('6ae5a411a3fe8b896ae3f9e2'),
    userId: 5,
    eventType: 'VIEW',
    propertyId: 3,
    metadata: {},
    createdAt: ISODate('2026-10-31T08:17:21.949Z')
  },
  {
    _id: ObjectId('6ae5a411a3fe8b896ae3f9e1'),
    userId: 5,
    eventType: 'VIEW',
    propertyId: 3,
    metadata: {},
    createdAt: ISODate('2026-10-31T08:17:21.880Z')
  },
  {
    _id: ObjectId('6ae5a228a3fe8b896ae3f9e0'),
    userId: 5,
    eventType: 'SEARCH',
    propertyId: null,
    metadata: { filters: { furnishing: 'UNFURNISHED' } },
    createdAt: ISODate('2026-10-31T08:09:12.847Z')
  },
  {
    _id: ObjectId('6ae5a225a3fe8b896ae3f9df'),
    userId: 5,
    eventType: 'SEARCH',
    propertyId: null,
    metadata: { filters: { furnishing: 'UNFURNISHED', amenityIds: '7' } },
    createdAt: ISODate('2026-10-31T08:09:09.088Z')
  },
  {
    _id: ObjectId('6ae5a17ca3fe8b896ae3f9de'),
    userId: 5,
    eventType: 'SEARCH',
    propertyId: null,
    metadata: { filters: { furnishing: 'UNFURNISHED' } },
    createdAt: ISODate('2026-10-31T08:06:20.090Z')
  },
  {
    _id: ObjectId('6ae59f3ba3fe8b896ae3f9dd'),
    userId: 5,
    eventType: 'VIEW',
    propertyId: 6,
    metadata: {},
    createdAt: ISODate('2026-10-31T07:56:43.019Z')
```

13 Application Screenshots

The web application is a Next.js interface. The navigation bar changes with the signed-in role: buyers see Browse, For you and Favourites; agents see Browse, My Properties and Dashboard.

13.1 Authentication

Figure 23 — Login page — email and password sign-in



13.2 Home page

Figure 24 — Home page hero — AI-powered property discovery (buyer view)

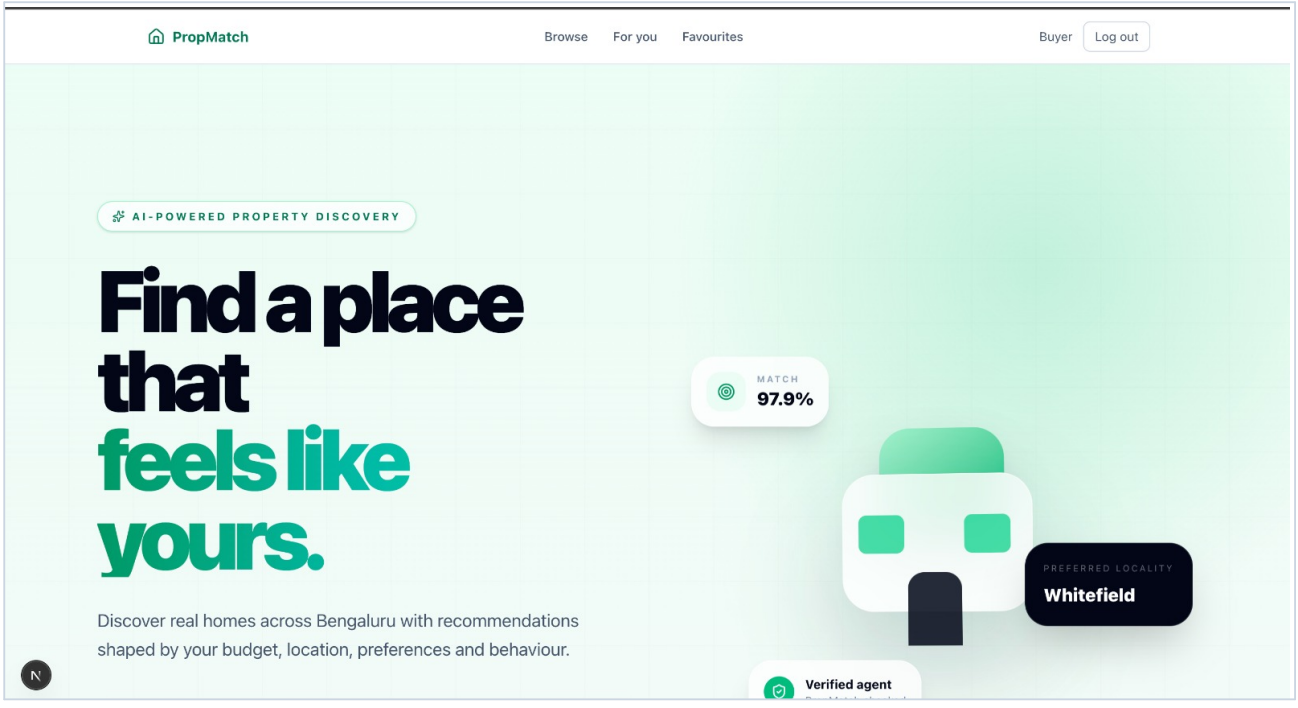


Figure 25 — Home page — featured property cards loaded from the PropMatch API

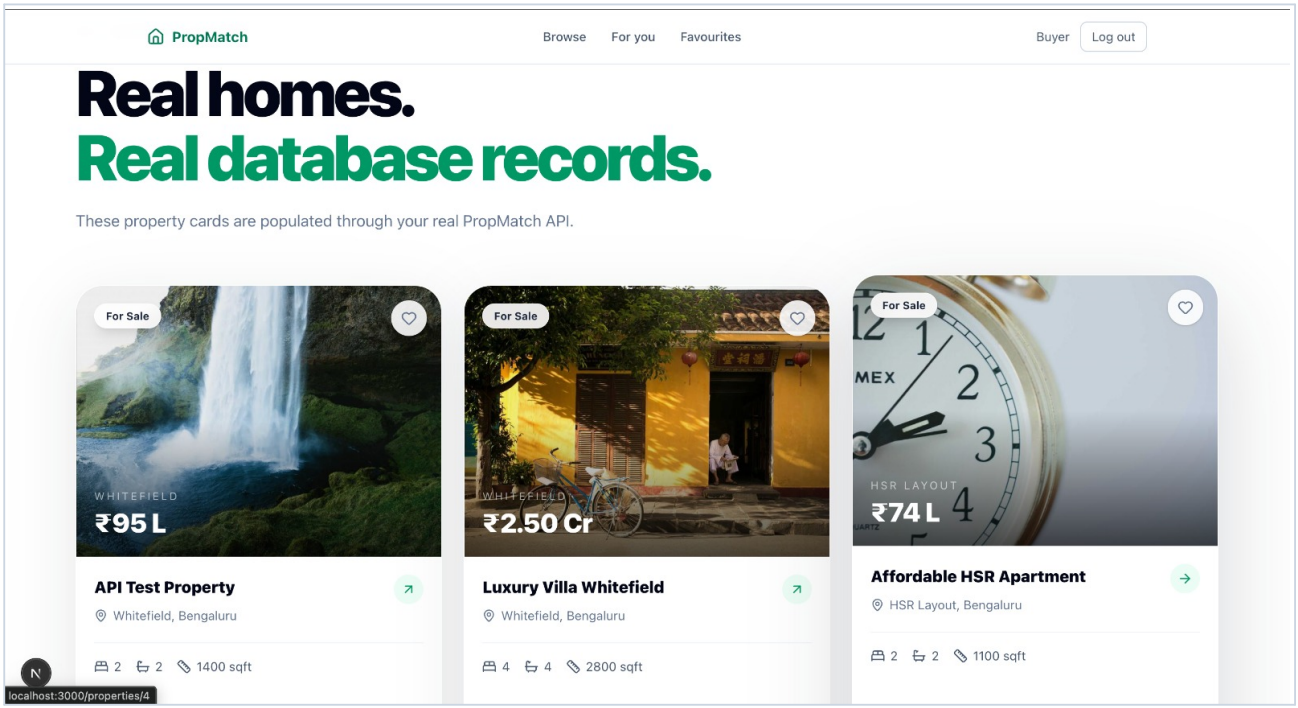


Figure 26 — Home page — multi-criteria search: budget, location and amenities

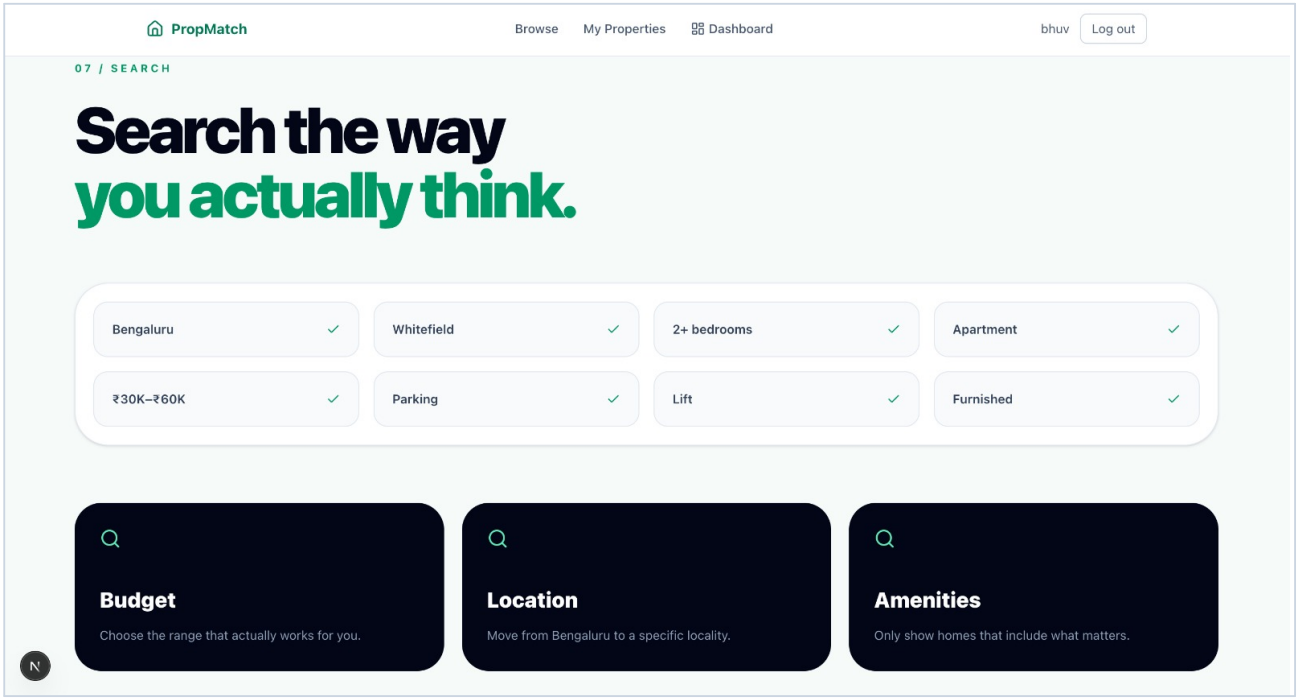


Figure 27 — Home page — “Keep exploring” property carousel

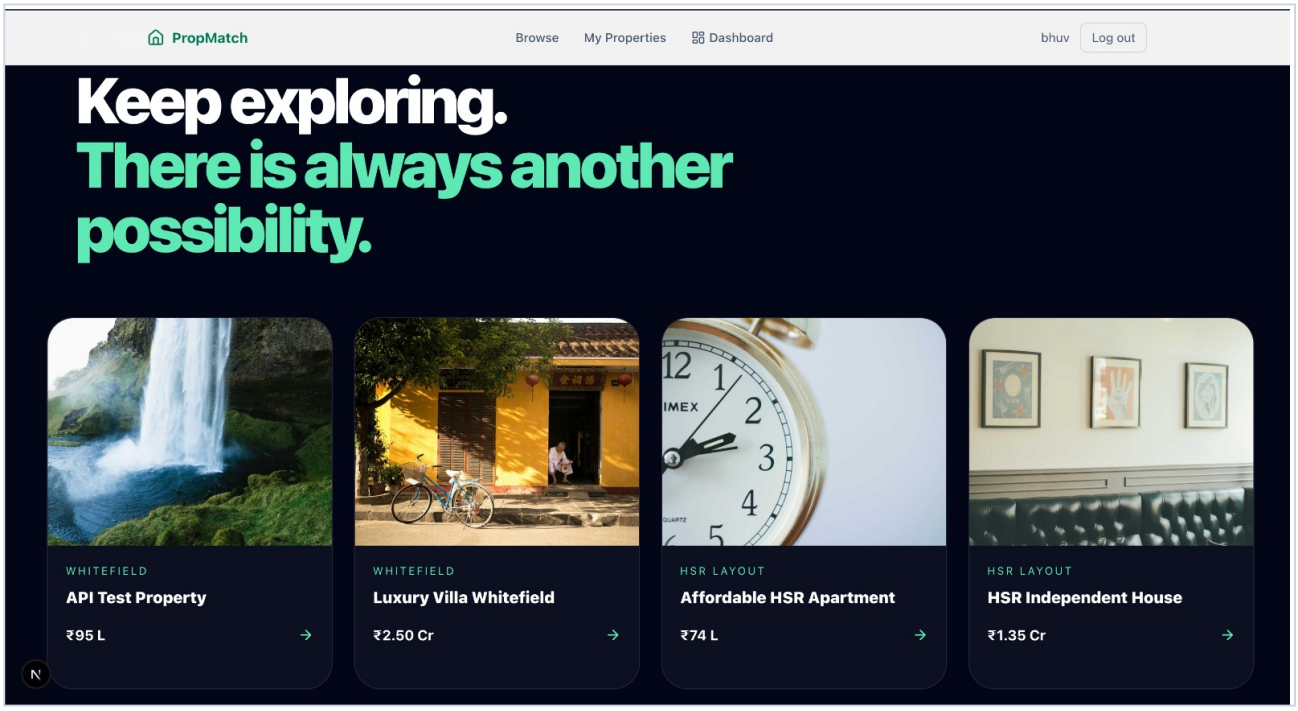
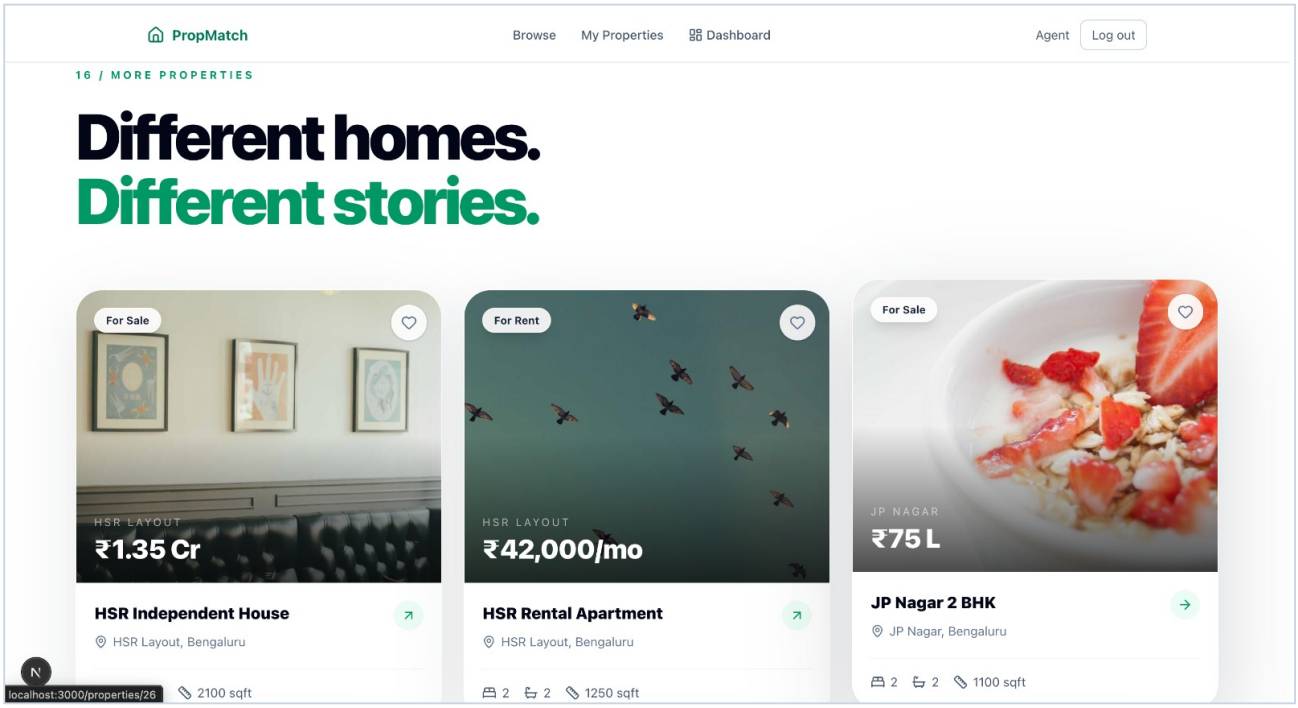


Figure 28 — Home page — more properties section



13.3 Buyer features

Figure 29 — Property detail page — images, specifications, agent contact, enquiry and site-visit actions

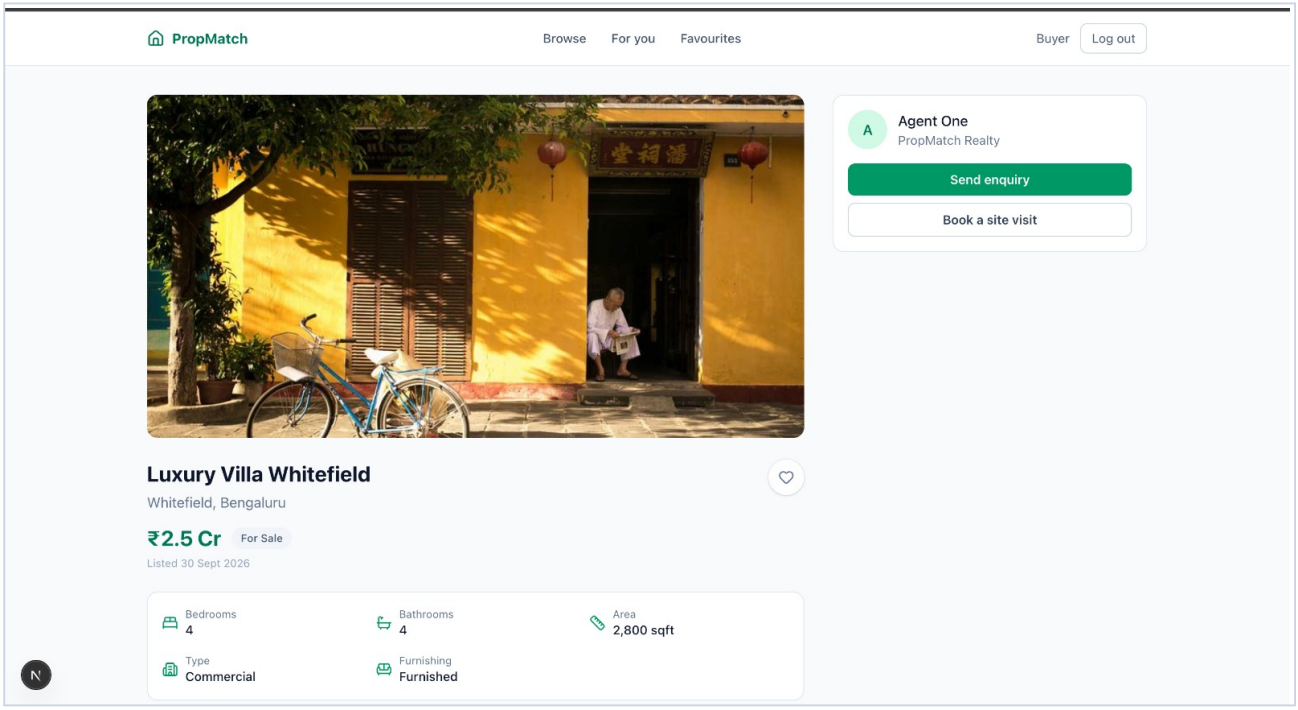


Figure 30 — For you — personalised recommendations with match % and reason tags

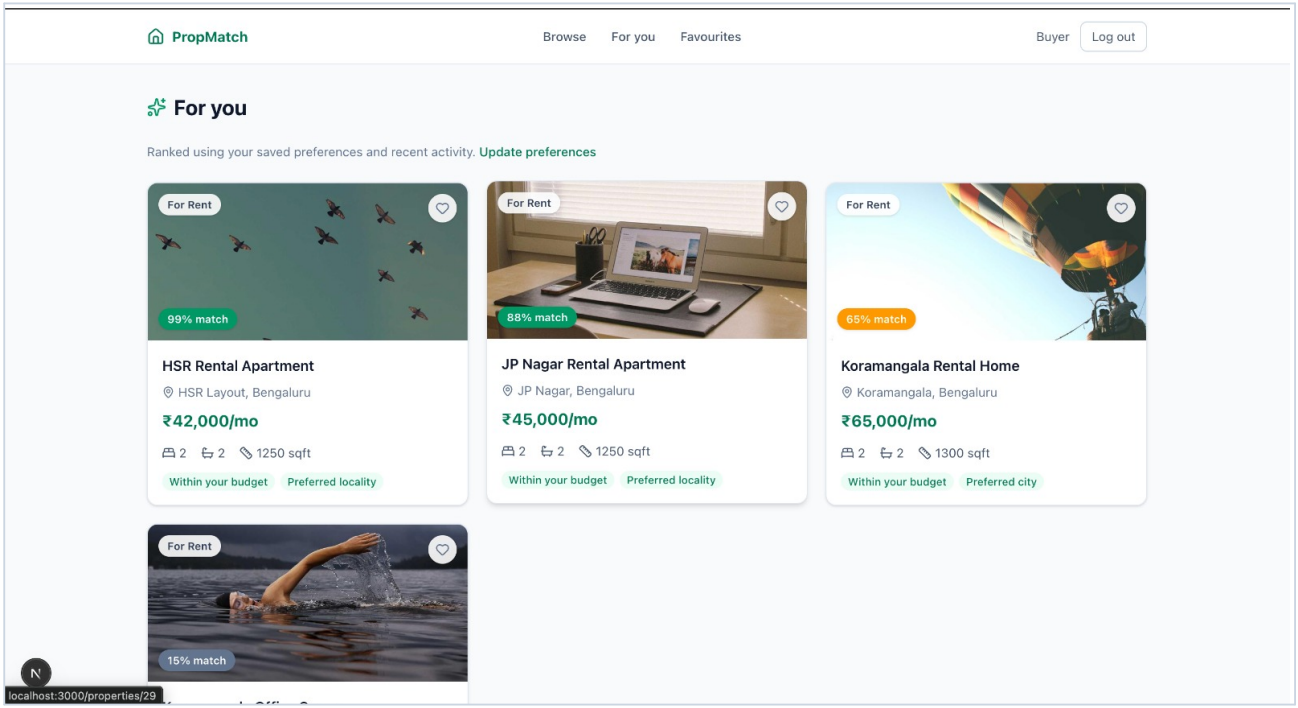
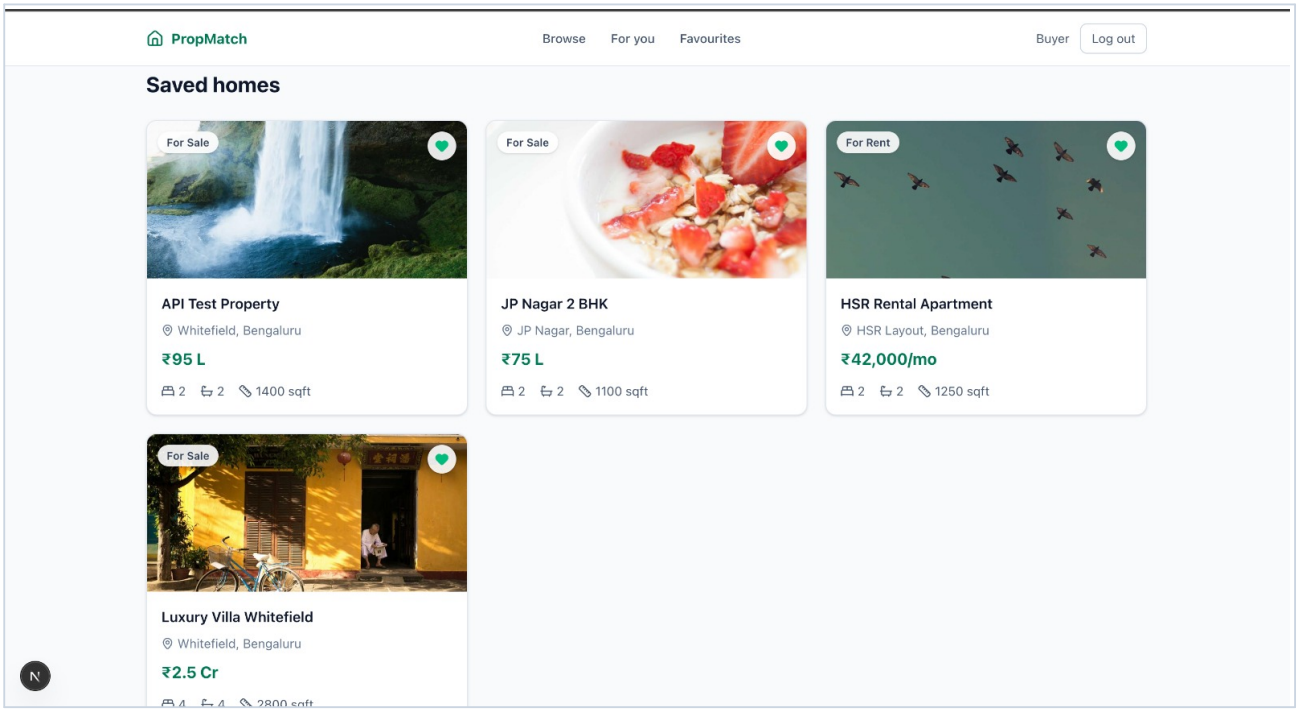


Figure 31 — Favourites — saved homes



13.4 Agent features

Figure 32 — My listings — edit, deactivate/reactivate and mark rented or sold

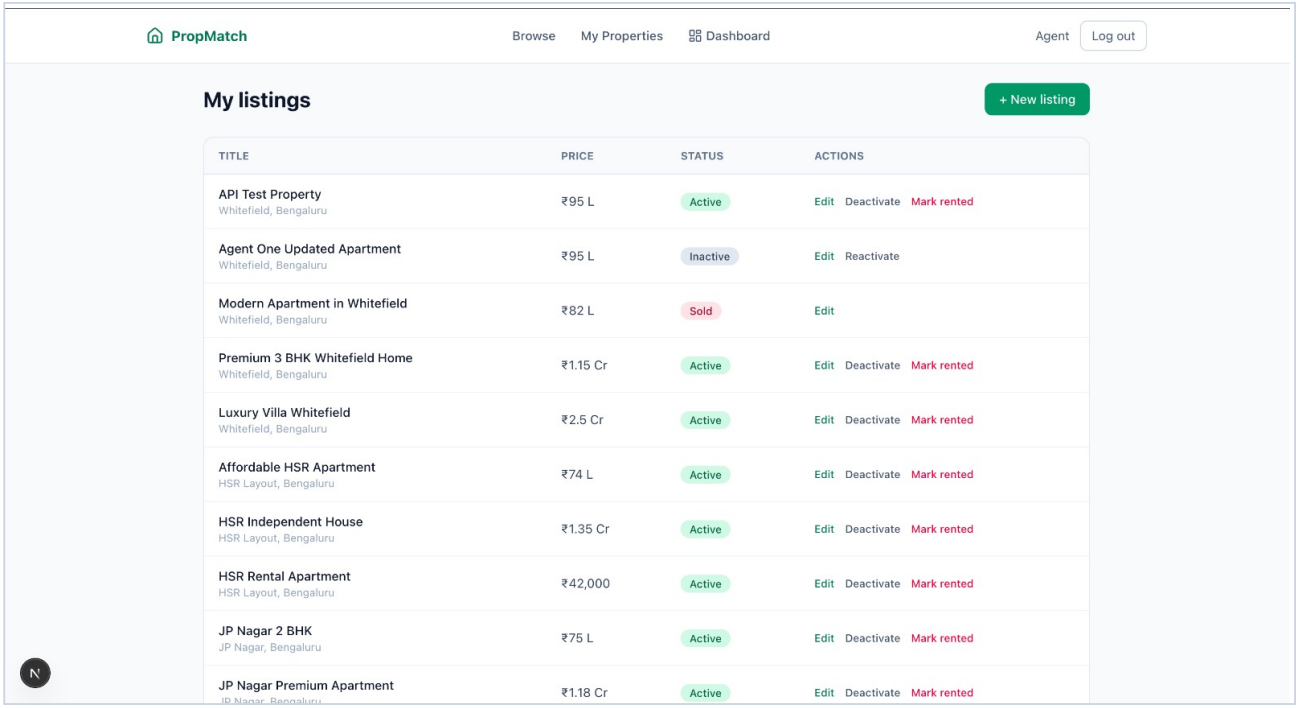
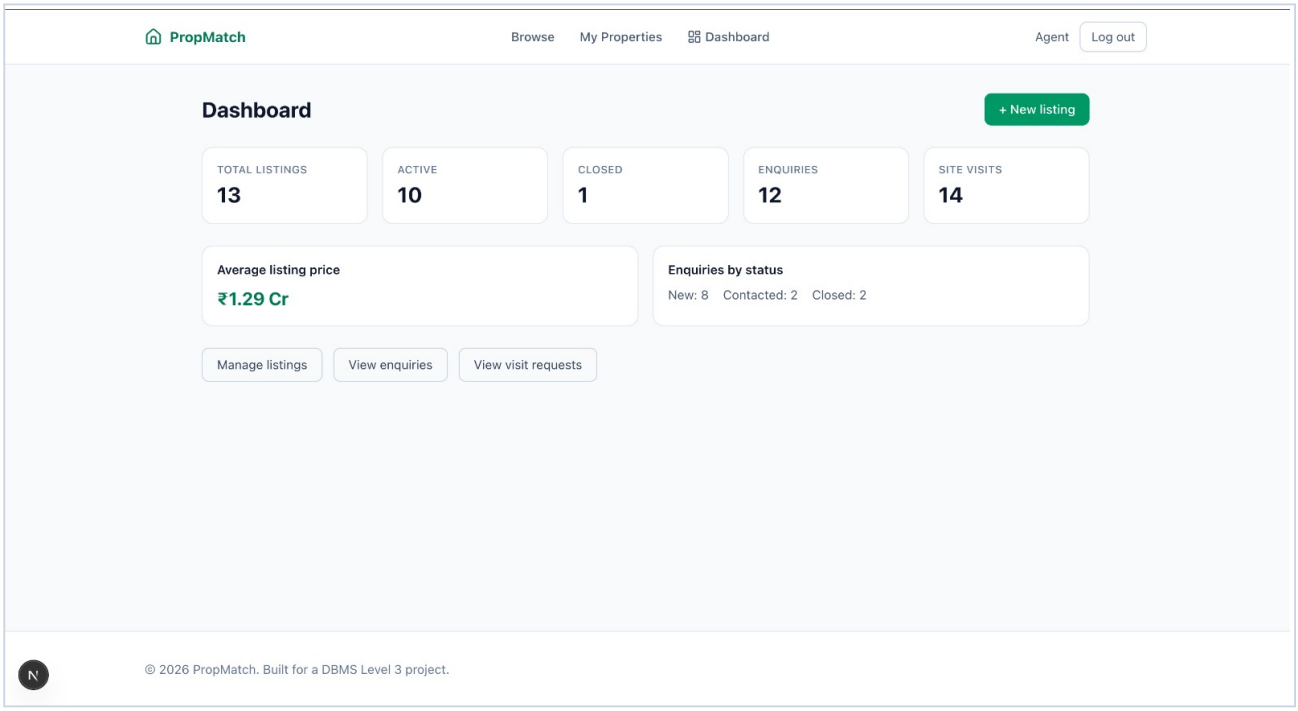


Figure 33 — Agent dashboard — listing, enquiry and site-visit statistics



14 Testing Results

Testing combined four kinds of evidence. Each is labelled so that it is clear what was run, where and how.

Level	What was done	Result
A. Database rules (executed)	32 test cases run against a MySQL 8.0.46 instance rebuilt from the propmatch schema dump (structure, triggers, procedures, function and views) with a small synthetic data set, on 1 November 2026.	32 of 32 passed
B. Live MySQL verification	Table listing, DESCRIBE, constraint metadata, record counts and the demonstrated SQL queries, captured from the project database (Section 12 and Figures 35–37).	All checks consistent
C. Live MongoDB verification	Document counts, distinct and aggregate queries, index listing and document inspection in propmatch_nosql (Section 12.2).	All checks consistent
D. Application walkthrough	Screenshots of buyer and agent screens (Section 13) cross-checked against database contents.	Behaviour matches the data

14.1 Database rule tests (level A)

The synthetic data set contained 4 users, 2 agent profiles (one verified), 2 locations, 2 property types and 4 properties. The “Actual result” column is the response returned by MySQL. These tests exercise the same constraints, triggers, procedures and views that exist in the project database; they do not use its data.

Constraints

ID	Test	Expected	Actual result (MySQL response)	Result
DB-01	Insert user with duplicate email	Rejected by UNIQUE uq_users_email	ERROR 1062: Duplicate entry 'buyer1@propmatch.test' for key 'users.uq_users_email'	PASS
DB-02	Insert user with malformed email	Rejected by CHECK chk_users_email	ERROR 3819: Check constraint 'chk_users_email' is violated.	PASS
DB-03	Insert user with invalid phone number	Rejected by CHECK chk_users_phone	ERROR 3819: Check constraint 'chk_users_phone' is violated.	PASS
DB-04	Insert user with invalid role value	Rejected (ENUM)	ERROR 1265: Data truncated for column 'role' at row 1	PASS
DB-05	Insert property with price = 0	Rejected by CHECK chk_prop_price	ERROR 3819: Check constraint 'chk_prop_price' is violated.	PASS
DB-06	Insert property with area = 0	Rejected by CHECK chk_prop_area	ERROR 3819: Check constraint 'chk_prop_area' is violated.	PASS
DB-07	Insert property for a non-existent agent	Rejected by FOREIGN KEY fk_prop_agent	ERROR 1452: Cannot add or update a child row: a foreign key constraint fails (`propmatch_test`.`properties`, CONSTRAIN	PASS
DB-08	Insert location with 5-character pincode	Rejected by CHECK chk_pincode	ERROR 3819: Check constraint 'chk_pincode' is violated.	PASS
DB-09	Insert location with latitude 95	Rejected by CHECK chk_lat	ERROR 3819: Check constraint 'chk_lat' is violated.	PASS
DB-10	Insert duplicate city + locality + pincode	Rejected by UNIQUE uq_location	ERROR 1062: Duplicate entry 'Bengaluru-Whitefield-560066' for key 'locations.uq_location'	PASS
DB-11	Insert the same favorite twice	Rejected by composite PRIMARY KEY	ERROR 1062: Duplicate entry '3-1' for key 'favorites.PRIMARY'	PASS
DB-12	Delete a user who has saved favorites	Rejected (FK RESTRICT on favorites.user_id)	ERROR 1451: Cannot delete or update a parent row: a foreign key constraint fails (`propmatch_test`.`favorites`, CONSTR	PASS

Triggers and function

ID	Test	Expected	Actual result (MySQL response)	Result
DB-13	Inquiry with a message shorter than 10 characters	Rejected by CHECK chk_inq_message	ERROR 3819: Check constraint 'chk_inq_message' is violated.	PASS
DB-14	Inquiry on an ACTIVE property	Accepted; status defaults to NEW	OK: NEW	PASS
DB-15	Inquiry on an INACTIVE property	Rejected by trigger: PROPERTY_NOT_AVAILABLE	ERROR 1644: PROPERTY_NOT_AVAILABLE	PASS
DB-16	Change a property's price	Trigger writes old/new price to price_history	OK: 8200000.00 -> 7900000.00	PASS
DB-17	Change a non-price column	No new price_history row	OK: 1	PASS
DB-18	Insert a site visit dated in the past	Rejected by trigger: VISIT_MUST_BE_IN_FUTURE	ERROR 1644: VISIT_MUST_BE_IN_FUTURE	PASS
DB-19	fn_price_per_sqft(8200000, 1200)	Returns 6833.33	OK: 6833.33	PASS
DB-20	fn_price_per_sqft with area = 0	Returns NULL, no division-by-zero error	OK: NULL	PASS

Stored procedures

ID	Test	Expected	Actual result (MySQL response)	Result
DB-21	sp_book_visit on an ACTIVE property	Visit created with status REQUESTED	OK: 1 REQUESTED	**PASS**
DB-22	sp_book_visit within 59 minutes of an existing booking	Rejected: SLOT_ALREADY_BOOKED	ERROR 1644: SLOT_ALREADY_BOOKED	**PASS**
DB-23	sp_book_visit 60+ minutes after the existing booking	Accepted (slots no longer conflict)	OK: 2	**PASS**
DB-24	sp_book_visit on an INACTIVE property	Rejected: PROPERTY_NOT_AVAILABLE	ERROR 1644: PROPERTY_NOT_AVAILABLE	**PASS**
DB-25	sp_close_property by an agent who does not own it	Rejected: NOT_PROPERTY_OWNER	ERROR 1644: NOT_PROPERTY_OWNER	**PASS**
DB-26	sp_close_property for a missing property	Rejected: PROPERTY_NOT_FOUND	ERROR 1644: PROPERTY_NOT_FOUND	**PASS**
DB-27	sp_close_property by the owner (SALE listing)	Status becomes SOLD; open visits CANCELLED; inquiries CLOSED	OK: SOLD / visits open: 0 / inquiries not closed: 0	**PASS**
DB-28	sp_close_property on a RENT listing	Status becomes RENTED	OK: RENTED	**PASS**

Views and cascades

ID	Test	Expected	Actual result (MySQL response)	Result
DB-29	v_active_listings contents	Only ACTIVE listings of verified agents (property 3 only)	OK: 3 8928.57 Villa	**PASS**
DB-30	v_agent_performance for agent 1	3 listings, 1 active, 2 closed, 1 inquiry, 2 visits (cancelled visits still counted)	OK: 3 1 2 1 2	**PASS**
DB-31	Delete an agent's user row	agent_profiles row removed by ON DELETE CASCADE (agent has no listings)	OK: 0	**PASS**
DB-32	Delete a user with login_audit rows	login_audit rows removed by ON DELETE CASCADE	OK: 0	**PASS**

Summary of level A

32 of 32 tests passed: 12 constraint tests, 8 trigger/function tests, 8 stored-procedure tests and 4 view/cascade tests. The rebuilt schema also reproduced the report's constraint totals exactly (13 PRIMARY KEY, 15 FOREIGN KEY, 5 UNIQUE, 10 CHECK).

14.2 Live MySQL verification (level B)

Check	Expected	Observed	Evidence
Objects in schema	13 tables + 2 views	15 objects listed by SHOW TABLES	Figure 35
Constraints	13 PK, 15 FK, 5 UNIQUE, 10 CHECK	43 rows in information_schema.TABLE_CONSTRAINTS	Figure 36
Column definitions	Types, NOT NULL, keys, defaults, ENUMs	DESCRIBE of users, properties and locations matches the data dictionary	Figure 34
Record counts	Main tables contain data	13 users, 32 properties, 8 locations, 17 inquiries, 31 images, 8 favorites, 18 visits	Figure 37
Status distribution	Sums to the property count	27 ACTIVE + 2 SOLD + 1 RENTED + 2 INACTIVE = 32	Figure 13
JOIN result	One row per property	32 rows with city and locality	Figure 15

14.3 Live MongoDB verification (level C)

Check	Expected	Observed	Evidence
Collections present	3 collections	user_activity 201, user_preferences 2, recommendations_cache 1 documents	Figure 18
Event types	Six types allowed by the validator	FAVORITE, INQUIRY, SEARCH, UNFAVORITE, VIEW, VISIT_REQUEST	Figure 19
Aggregation total	Group totals add up to the document count	77 + 56 + 29 + 26 + 8 + 5 = 201	Figure 20
Indexes	As defined in the setup script	Compound, unique and TTL indexes present (15552000 s and 3600 s)	Figure 8
Cache structure	items array with score and reasons	Each item has score, matchScore and a reasons array	Figures 6–7

14.4 Application walkthrough and data cross-checks (level D)

Feature	Observed in application	Cross-check against the databases	Result
Login (FR-01)	Buyer sign-in form accepts email and password (Fig. 23).	Buyer account exists in users with role BUYER.	Consistent
Listing cards (FR-02)	Cards show price, locality, beds, baths, area; e.g. API Test Property ₹95 L (Fig. 25).	Price 9500000.00 in the ORDER BY output equals ₹95 L.	Consistent
Recommendations (FR-08)	For you shows HSR Rental Apartment 99%, JP Nagar Rental Apartment 88%, Koramangala Rental Home 65% and a 15% match (Fig. 30).	recommendations_cache items for user 5 list property 6 (99.1), 29 (87.6), 13 (65) and 14 (15) in the same order, with the same reason tags.	Consistent
Favourites (FR-05)	Saved homes page lists four properties (Fig. 31).	FAVORITE / UNFAVORITE events for the same buyer are recorded in user_activity (Fig. 22).	Consistent
Agent dashboard (FR-10)	13 listings, 10 active, 1 closed, 12 enquiries, 14 visits, average ₹1.29 Cr (Fig. 33).	Identical to the Agent One row of v_agent_performance: 13, 10, 1, 12, 14, 12,877,312.50.	Consistent
Listing management (FR-09)	My listings shows Active, Inactive and Sold statuses with the matching actions (Fig. 32).	Statuses correspond to the properties.status values (27 ACTIVE, 2 SOLD, 1 RENTED, 2 INACTIVE).	Consistent

14.5 Observations and limitations

The following points were noticed while testing. They are not failures of the tests above, but they should be known when presenting the project.

- **No automated end-to-end test suite was supplied.** The database layer is verified by level A; user interface behaviour is evidenced by screenshots and cross-checks, not by scripted browser tests.
- **Vector search is not implemented.** The word “vector” should not be used to describe the recommendation component (Section 11.5).
- **MongoDB validators run in warn mode.** Non-conforming documents are accepted with a warning. The preference document for user 6 still uses the nested seed shape, and no createdAt field is visible on the user 5 document.
- **Budget tolerance in recommendations.** Koramangala Rental Home is rent ₹65,000 yet is tagged “Within your budget” while the stored maximum budget is ₹60,000, so the scorer appears to allow some tolerance above the maximum. This should be confirmed in the recommender code.
- **v_agent_performance counts cancelled visits.** In test DB-30 the two visits cancelled by sp_close_property are still included in total_visits, because the view counts every visit row regardless of status.
- **Test data versus project data.** Level A used synthetic data on a rebuilt schema; project record counts and query outputs come from the screenshots in Sections 10 and 12.

14.6 Supporting verification screenshots

Figure 34 — DESCRIBE — column definitions of users, properties and locations

Data types, NOT NULL settings, keys, defaults, auto-increment and ENUM columns.

```
mysql> DESCRIBE users;
+---------------+-----------------------------------+------+-----+-------------------+-----------------------------------------------+
| Field         | Type                              | Null | Key | Default           | Extra                                         |
+---------------+-----------------------------------+------+-----+-------------------+-----------------------------------------------+
| user_id       | int unsigned                      | NO   | PRI | NULL              | auto_increment                                |
| full_name     | varchar(100)                      | NO   |     | NULL              |                                               |
| email         | varchar(150)                      | NO   | UNI | NULL              |                                               |
| password_hash | varchar(255)                      | NO   |     | NULL              |                                               |
| phone         | varchar(15)                       | YES  |     | NULL              |                                               |
| role          | enum('BUYER','AGENT','ADMIN')     | NO   |     | BUYER             |                                               |
| is_active     | tinyint(1)                        | NO   |     | 1                 |                                               |
| created_at    | timestamp                         | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED                             |
| updated_at    | timestamp                         | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED on update CURRENT_TIMESTAMP |
+---------------+-----------------------------------+------+-----+-------------------+-----------------------------------------------+
9 rows in set (0.005 sec)

mysql> DESCRIBE properties;
+--------------+-----------------------------------------------+------+-----+-------------------+-----------------------------------------------+
| Field        | Type                                          | Null | Key | Default           | Extra                                         |
+--------------+-----------------------------------------------+------+-----+-------------------+-----------------------------------------------+
| property_id  | int unsigned                                  | NO   | PRI | NULL              | auto_increment                                |
| agent_id     | int unsigned                                  | NO   | MUL | NULL              |                                               |
| location_id  | int unsigned                                  | NO   | MUL | NULL              |                                               |
| type_id      | int unsigned                                  | NO   | MUL | NULL              |                                               |
| title        | varchar(150)                                  | NO   | MUL | NULL              |                                               |
| description  | text                                          | NO   |     | NULL              |                                               |
| listing_type | enum('SALE','RENT')                           | NO   |     | NULL              |                                               |
| price        | decimal(14,2)                                 | NO   |     | NULL              |                                               |
| area_sqft    | int unsigned                                  | NO   |     | NULL              |                                               |
| bedrooms     | tinyint unsigned                              | NO   |     | 0                 |                                               |
| bathrooms    | tinyint unsigned                              | NO   |     | 0                 |                                               |
| furnishing   | enum('UNFURNISHED','SEMI_FURNISHED','FURNISHED') | NO   |     | UNFURNISHED       |                                               |
| address_line | varchar(200)                                  | YES  |     | NULL              |                                               |
| status       | enum('ACTIVE','SOLD','RENTED','INACTIVE')     | NO   | MUL | ACTIVE            |                                               |
| created_at   | timestamp                                     | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED                             |
| updated_at   | timestamp                                     | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED on update CURRENT_TIMESTAMP |
+--------------+-----------------------------------------------+------+-----+-------------------+-----------------------------------------------+
16 rows in set (0.012 sec)

mysql> DESCRIBE locations;
+-------------+--------------+------+-----+---------+----------------+
| Field       | Type         | Null | Key | Default | Extra          |
+-------------+--------------+------+-----+---------+----------------+
| location_id | int unsigned | NO   | PRI | NULL    | auto_increment |
| city        | varchar(80)  | NO   | MUL | NULL    |                |
| locality    | varchar(100) | NO   |     | NULL    |                |
| state       | varchar(80)  | NO   |     | NULL    |                |
| pincode     | char(6)      | NO   |     | NULL    |                |
| latitude    | decimal(9,6) | YES  |     | NULL    |                |
| longitude   | decimal(9,6) | YES  |     | NULL    |                |
+-------------+--------------+------+-----+---------+----------------+
7 rows in set (0.002 sec)
```

Figure 35 — SHOW TABLES — 13 tables and 2 views

Lists 15 objects in the propmatch database.

```
mysql> SHOW TABLES;
+----------------------+
| Tables_in_propmatch  |
+----------------------+
| agent_profiles       |
| amenities            |
| favorites            |
| inquiries            |
| locations            |
| login_audit          |
| price_history        |
| properties           |
| property_amenities   |
| property_images      |
| property_types       |
| site_visits          |
| users                |
| v_active_listings    |
| v_agent_performance  |
+----------------------+
15 rows in set (0.014 sec)
```

Figure 36 — Constraint metadata — information_schema.TABLE_CONSTRAINTS

Returns 43 constraints: 13 PRIMARY KEY, 15 FOREIGN KEY, 5 UNIQUE and 10 CHECK.

```
mysql> SELECT TABLE_NAME, CONSTRAINT_NAME, CONSTRAINT_TYPE
    -> FROM information_schema.TABLE_CONSTRAINTS
    -> WHERE TABLE_SCHEMA = 'propmatch'
    -> ORDER BY TABLE_NAME, CONSTRAINT_TYPE;
+--------------------+---------------------+-----------------+
| TABLE_NAME         | CONSTRAINT_NAME     | CONSTRAINT_TYPE |
+--------------------+---------------------+-----------------+
| agent_profiles     | fk_agent_user       | FOREIGN KEY     |
| agent_profiles     | PRIMARY             | PRIMARY KEY     |
| agent_profiles     | uq_agent_license    | UNIQUE          |
| amenities          | PRIMARY             | PRIMARY KEY     |
| amenities          | uq_amenity_name     | UNIQUE          |
| favorites          | fk_fav_property     | FOREIGN KEY     |
| favorites          | fk_fav_user         | FOREIGN KEY     |
| favorites          | PRIMARY             | PRIMARY KEY     |
| inquiries          | chk_inq_message     | CHECK           |
| inquiries          | fk_inq_buyer        | FOREIGN KEY     |
| inquiries          | fk_inq_property     | FOREIGN KEY     |
| inquiries          | PRIMARY             | PRIMARY KEY     |
| locations          | chk_lat             | CHECK           |
| locations          | chk_lng             | CHECK           |
| locations          | chk_pincode         | CHECK           |
| locations          | PRIMARY             | PRIMARY KEY     |
| locations          | uq_location         | UNIQUE          |
| login_audit        | fk_login_audit_user | FOREIGN KEY     |
| login_audit        | PRIMARY             | PRIMARY KEY     |
| price_history      | fk_ph_property      | FOREIGN KEY     |
| price_history      | PRIMARY             | PRIMARY KEY     |
| properties         | chk_prop_price      | CHECK           |
| properties         | chk_prop_area       | CHECK           |
| properties         | chk_prop_baths      | CHECK           |
| properties         | chk_prop_bedrooms   | CHECK           |
| properties         | fk_prop_agent       | FOREIGN KEY     |
| properties         | fk_prop_location    | FOREIGN KEY     |
| properties         | fk_prop_type        | FOREIGN KEY     |
| properties         | PRIMARY             | PRIMARY KEY     |
| property_amenities | fk_pa_amenity       | FOREIGN KEY     |
| property_amenities | fk_pa_property      | FOREIGN KEY     |
| property_amenities | PRIMARY             | PRIMARY KEY     |
| property_images    | fk_img_property     | FOREIGN KEY     |
| property_images    | PRIMARY             | PRIMARY KEY     |
| property_types     | PRIMARY             | PRIMARY KEY     |
| property_types     | uq_type_name        | UNIQUE          |
| site_visits        | fk_visit_buyer      | FOREIGN KEY     |
| site_visits        | fk_visit_property   | FOREIGN KEY     |
| site_visits        | PRIMARY             | PRIMARY KEY     |
| users              | chk_users_email     | CHECK           |
| users              | chk_users_phone     | CHECK           |
| users              | PRIMARY             | PRIMARY KEY     |
| users              | uq_users_email      | UNIQUE          |
+--------------------+---------------------+-----------------+
43 rows in set (0.003 sec)
```

Figure 37 — Record counts — COUNT(*) per table

13 users, 32 properties, 8 locations, 17 inquiries, 31 property images, 8 favorites and 18 site visits.

```
mysql> SELECT COUNT(*) AS total_users FROM users;
+-------------+
| total_users |
+-------------+
|          13 |
+-------------+
1 row in set (0.001 sec)

mysql> SELECT COUNT(*) AS total_properties FROM properties;
+------------------+
| total_properties |
+------------------+
|               32 |
+------------------+
1 row in set (0.000 sec)

mysql> SELECT COUNT(*) AS total_locations FROM locations;
+-----------------+
| total_locations |
+-----------------+
|               8 |
+-----------------+
1 row in set (0.000 sec)

mysql> SELECT COUNT(*) AS total_inquiries FROM inquiries;
+-----------------+
| total_inquiries |
+-----------------+
|              17 |
+-----------------+
1 row in set (0.000 sec)

mysql> SELECT COUNT(*) AS total_images FROM property_images;
+--------------+
| total_images |
+--------------+
|           31 |
+--------------+
1 row in set (0.000 sec)

mysql> SELECT COUNT(*) AS total_favorites FROM favorites;
+-----------------+
| total_favorites |
+-----------------+
|               8 |
+-----------------+
1 row in set (0.000 sec)

mysql> SELECT COUNT(*) AS total_site_visits FROM site_visits;
+-------------------+
| total_site_visits |
+-------------------+
|                18 |
+-------------------+
1 row in set (0.000 sec)
```

15 SQL – Database Commands

This section lists the SQL commands used to build and use the propmatch database. Triggers, the function, stored procedures and views are listed in Section 10; the demonstrated SELECT queries are in Section 12.

15.1 Database and table creation (DDL)

```
CREATE DATABASE propmatch CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE propmatch;
```

```
CREATE TABLE users (
  user_id int unsigned NOT NULL AUTO_INCREMENT,
  full_name varchar(100) NOT NULL,
  email varchar(150) NOT NULL,
  password_hash varchar(255) NOT NULL,
  phone varchar(15) DEFAULT NULL,
  role enum('BUYER','AGENT','ADMIN') NOT NULL DEFAULT 'BUYER',
  is_active tinyint(1) NOT NULL DEFAULT '1',
  created_at timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (user_id),
  UNIQUE KEY uq_users_email (email),
  CONSTRAINT chk_users_email CHECK ((email like _utf8mb4'%_@_%._%')),
  CONSTRAINT chk_users_phone CHECK (((phone is null) or regexp_like(phone,_utf8mb4'^[0-9+ -]{10,15}$')))
) ENGINE=InnoDB;
```

```
CREATE TABLE agent_profiles (
  agent_id int unsigned NOT NULL,
  agency_name varchar(120) NOT NULL,
  license_no varchar(50) NOT NULL,
  bio text,
  is_verified tinyint(1) NOT NULL DEFAULT '0',
  verified_at timestamp NULL DEFAULT NULL,
  PRIMARY KEY (agent_id),
  UNIQUE KEY uq_agent_license (license_no),
  CONSTRAINT fk_agent_user FOREIGN KEY (agent_id) REFERENCES users (user_id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

```
CREATE TABLE amenities (
  amenity_id int unsigned NOT NULL AUTO_INCREMENT,
  amenity_name varchar(60) NOT NULL,
  PRIMARY KEY (amenity_id),
  UNIQUE KEY uq_amenity_name (amenity_name)
) ENGINE=InnoDB;
```

```
CREATE TABLE locations (
  location_id int unsigned NOT NULL AUTO_INCREMENT,
  city varchar(80) NOT NULL,
  locality varchar(100) NOT NULL,
  state varchar(80) NOT NULL,
  pincode char(6) NOT NULL,
  latitude decimal(9,6) DEFAULT NULL,
  longitude decimal(9,6) DEFAULT NULL,
  PRIMARY KEY (location_id),
  UNIQUE KEY uq_location (city,locality,pincode),
  CONSTRAINT chk_lat CHECK (((latitude is null) or (latitude between -(90) and 90))),
  CONSTRAINT chk_lng CHECK (((longitude is null) or (longitude between -(180) and 180))),
  CONSTRAINT chk_pincode CHECK (regexp_like(pincode,_utf8mb4'^[0-9]{6}$'))
) ENGINE=InnoDB;
```

```
CREATE TABLE property_types (
  type_id int unsigned NOT NULL AUTO_INCREMENT,
  type_name varchar(50) NOT NULL,
  PRIMARY KEY (type_id),
  UNIQUE KEY uq_type_name (type_name)
) ENGINE=InnoDB;
```

```
CREATE TABLE properties (
  property_id int unsigned NOT NULL AUTO_INCREMENT,
  agent_id int unsigned NOT NULL,
  location_id int unsigned NOT NULL,
  type_id int unsigned NOT NULL,
  title varchar(150) NOT NULL,
  description text NOT NULL,
  listing_type enum('SALE','RENT') NOT NULL,
  price decimal(14,2) NOT NULL,
  area_sqft int unsigned NOT NULL,
  bedrooms tinyint unsigned NOT NULL DEFAULT '0',
  bathrooms tinyint unsigned NOT NULL DEFAULT '0',
  furnishing enum('UNFURNISHED','SEMI_FURNISHED','FURNISHED') NOT NULL DEFAULT 'UNFURNISHED',
  address_line varchar(200) DEFAULT NULL,
  status enum('ACTIVE','SOLD','RENTED','INACTIVE') NOT NULL DEFAULT 'ACTIVE',
  created_at timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (property_id),
  KEY fk_prop_location (location_id),
  KEY idx_prop_search (status,listing_type,location_id,price),
  KEY idx_prop_agent (agent_id),
  KEY idx_prop_type (type_id),
  FULLTEXT KEY ftx_prop_text (title,description),
  CONSTRAINT fk_prop_agent FOREIGN KEY (agent_id) REFERENCES agent_profiles (agent_id),
  CONSTRAINT fk_prop_location FOREIGN KEY (location_id) REFERENCES locations (location_id),
  CONSTRAINT fk_prop_type FOREIGN KEY (type_id) REFERENCES property_types (type_id),
  CONSTRAINT chk_prop_area CHECK ((area_sqft > 0)),
  CONSTRAINT chk_prop_baths CHECK ((bathrooms <= 20)),
  CONSTRAINT chk_prop_bedrooms CHECK ((bedrooms <= 50)),
  CONSTRAINT chk_prop_price CHECK ((price > 0))
) ENGINE=InnoDB;
```

```
CREATE TABLE favorites (
  user_id int unsigned NOT NULL,
  property_id int unsigned NOT NULL,
  saved_at timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (user_id,property_id),
  KEY fk_fav_property (property_id),
  CONSTRAINT fk_fav_property FOREIGN KEY (property_id) REFERENCES properties (property_id),
  CONSTRAINT fk_fav_user FOREIGN KEY (user_id) REFERENCES users (user_id)
) ENGINE=InnoDB;
```

```
CREATE TABLE inquiries (
  inquiry_id int unsigned NOT NULL AUTO_INCREMENT,
  property_id int unsigned NOT NULL,
  buyer_id int unsigned NOT NULL,
  message varchar(1000) NOT NULL,
  status enum('NEW','CONTACTED','CLOSED') NOT NULL DEFAULT 'NEW',
  created_at timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (inquiry_id),
  KEY idx_inq_property (property_id,status),
  KEY idx_inq_buyer (buyer_id),
  CONSTRAINT fk_inq_buyer FOREIGN KEY (buyer_id) REFERENCES users (user_id),
  CONSTRAINT fk_inq_property FOREIGN KEY (property_id) REFERENCES properties (property_id),
  CONSTRAINT chk_inq_message CHECK ((char_length(trim(message)) >= 10))
) ENGINE=InnoDB;
```

```
CREATE TABLE login_audit (
  login_id bigint unsigned NOT NULL AUTO_INCREMENT,
  user_id int unsigned NOT NULL,
  login_at timestamp(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  ip_address varchar(45) DEFAULT NULL,
  user_agent varchar(500) DEFAULT NULL,
  PRIMARY KEY (login_id),
  KEY idx_login_audit_user_time (user_id,login_at DESC),
  KEY idx_login_audit_time (login_at DESC),
  CONSTRAINT fk_login_audit_user FOREIGN KEY (user_id) REFERENCES users (user_id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

```
CREATE TABLE price_history (
  history_id int unsigned NOT NULL AUTO_INCREMENT,
  property_id int unsigned NOT NULL,
  old_price decimal(14,2) NOT NULL,
  new_price decimal(14,2) NOT NULL,
  changed_at timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (history_id),
  KEY idx_ph_property (property_id,changed_at),
  CONSTRAINT fk_ph_property FOREIGN KEY (property_id) REFERENCES properties (property_id)
) ENGINE=InnoDB;
```

```
CREATE TABLE property_amenities (
  property_id int unsigned NOT NULL,
  amenity_id int unsigned NOT NULL,
  PRIMARY KEY (property_id,amenity_id),
  KEY fk_pa_amenity (amenity_id),
  CONSTRAINT fk_pa_amenity FOREIGN KEY (amenity_id) REFERENCES amenities (amenity_id),
  CONSTRAINT fk_pa_property FOREIGN KEY (property_id) REFERENCES properties (property_id)
) ENGINE=InnoDB;
```

```
CREATE TABLE property_images (
  image_id int unsigned NOT NULL AUTO_INCREMENT,
  property_id int unsigned NOT NULL,
  image_url varchar(500) NOT NULL,
  is_primary tinyint(1) NOT NULL DEFAULT '0',
  PRIMARY KEY (image_id),
  KEY idx_img_property (property_id),
  CONSTRAINT fk_img_property FOREIGN KEY (property_id) REFERENCES properties (property_id)
) ENGINE=InnoDB;
```

```
CREATE TABLE site_visits (
  visit_id int unsigned NOT NULL AUTO_INCREMENT,
  property_id int unsigned NOT NULL,
  buyer_id int unsigned NOT NULL,
  scheduled_at datetime NOT NULL,
  status enum('REQUESTED','CONFIRMED','COMPLETED','CANCELLED') NOT NULL DEFAULT 'REQUESTED',
  created_at timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (visit_id),
  KEY idx_visit_slot (property_id,scheduled_at),
  KEY idx_visit_buyer (buyer_id),
  CONSTRAINT fk_visit_buyer FOREIGN KEY (buyer_id) REFERENCES users (user_id),
  CONSTRAINT fk_visit_property FOREIGN KEY (property_id) REFERENCES properties (property_id)
) ENGINE=InnoDB;
```

15.2 Inserting, updating and deleting data (DML)

Representative statements for every main table. They were executed successfully against a fresh copy of the schema; the UPDATE of the price fires trg_properties_price_history, and sp_close_property cancels open visits and closes open enquiries.

```
-- Reference data
INSERT INTO property_types (type_name) VALUES ('Apartment'), ('Independent House'), ('Villa'), ('Plot'),
    ('Commercial');
INSERT INTO amenities (amenity_name) VALUES ('Parking'), ('Lift'), ('Gym'), ('Security');
INSERT INTO locations (city, locality, state, pincode, latitude, longitude)
VALUES ('Bengaluru', 'Whitefield', 'Karnataka', '560066', 12.969800, 77.750000),
       ('Bengaluru', 'HSR Layout', 'Karnataka', '560102', 12.911600, 77.638900);

-- Users: one agent (with profile) and one buyer
INSERT INTO users (full_name, email, password_hash, phone, role)
VALUES ('Agent One', 'agent1@propmatch.test', '$2b$10$exampleHashValue', '9876543210', 'AGENT'),
       ('Buyer One', 'buyer1@propmatch.test', '$2b$10$exampleHashValue', '9876543212', 'BUYER');
INSERT INTO agent_profiles (agent_id, agency_name, license_no, bio, is_verified, verified_at)
VALUES (1, 'PropMatch Realty', 'LIC-001', 'Residential specialist', 1, NOW());

-- Listings, images and amenities
INSERT INTO properties (agent_id, location_id, type_id, title, description, listing_type, price, area_sqft,
    bedrooms, bathrooms, furnishing)
VALUES (1, 1, 1, 'Modern Apartment in Whitefield', 'Spacious 2 BHK apartment with excellent connectivity.',
    'SALE', 8200000, 1200, 2, 2, 'FURNISHED'),
       (1, 2, 1, 'HSR Rental Apartment', 'Well maintained 2 BHK apartment close to restaurants.', 'RENT',
           42000, 1250, 2, 2, 'SEMI_FURNISHED');
INSERT INTO property_images (property_id, image_url, is_primary) VALUES (1,
    'https://picsum.photos/seed/propmatch1/800/600', 1);
INSERT INTO property_amenities (property_id, amenity_id) VALUES (1, 1), (1, 2), (2, 1);

-- Buyer activity
INSERT INTO inquiries (property_id, buyer_id, message) VALUES (1, 2, 'Is this apartment still available for
    viewing?');
INSERT INTO favorites (user_id, property_id) VALUES (2, 1);
CALL sp_book_visit(1, 2, '2026-12-15 11:00:00', @visit_id);

-- UPDATE: a price change is logged automatically by trg_properties_price_history
UPDATE properties SET price = 7900000 WHERE property_id = 1;
-- UPDATE: deactivate a listing
UPDATE properties SET status = 'INACTIVE' WHERE property_id = 2;
-- UPDATE: agent follows up on an enquiry
UPDATE inquiries SET status = 'CONTACTED' WHERE inquiry_id = 1;
-- Close a sold listing (cancels open visits, closes enquiries)
CALL sp_close_property(1, 1, FALSE);

-- DELETE: remove a favourite and an image
DELETE FROM favorites WHERE user_id = 2 AND property_id = 1;
DELETE FROM property_images WHERE image_id = 1;
```

15.3 Other commands used

```
SHOW TABLES;                                  -- 13 tables + 2 views (Figure 35)
DESCRIBE users;  DESCRIBE properties;  DESCRIBE locations;      -- column definitions (Figure 34)
SELECT TABLE_NAME, CONSTRAINT_NAME, CONSTRAINT_TYPE
FROM information_schema.TABLE_CONSTRAINTS
WHERE TABLE_SCHEMA = 'propmatch' ORDER BY TABLE_NAME, CONSTRAINT_TYPE;   -- 43 rows (Figure 36)
SELECT COUNT(*) AS total_users FROM users;     -- same pattern for each table (Figure 37)
SELECT * FROM v_active_listings LIMIT 10;      SELECT * FROM v_agent_performance;
```

16 NoSQL / Vector Database – Commands

These are the MongoDB shell (mongosh) commands for database propmatch_nosql. The index and query commands are exactly those shown in the screenshots in Sections 11 and 12.

16.1 Database, validation and index setup

```
use propmatch_nosql

// Collections with JSON Schema validators (moderate level, warn action)
db.createCollection("user_preferences", {
  validator: { $jsonSchema: { bsonType: "object",
    required: ["userId","listingType","budgetMin","budgetMax","preferredCities","preferredLocalities",
               "preferredTypeIds","minBedrooms","furnishing","mustHaveAmenityIds","createdAt","updatedAt"] }
                   },
  validationLevel: "moderate", validationAction: "warn" })

db.createCollection("user_activity", {
  validator: { $jsonSchema: { bsonType: "object", required: ["userId","eventType","createdAt"],
    properties: { eventType: { enum: ["VIEW","SEARCH","FAVORITE","UNFAVORITE","INQUIRY","VISIT_REQUEST"] } }
       } },
  validationLevel: "moderate", validationAction: "warn" })

db.createCollection("recommendations_cache", {
  validator: { $jsonSchema: { bsonType: "object", required: ["userId","items","generatedAt"],
    properties: { items: { bsonType: "array", items: { bsonType: "object",
      required: ["propertyId","score","reasons"], properties: { reasons: { bsonType: "array", items: {
          bsonType: "string" } } } } } } } },
  validationLevel: "moderate", validationAction: "warn" })

// Indexes
db.user_preferences.createIndex({ userId: 1 }, { unique: true })
db.user_activity.createIndex({ userId: 1, createdAt: -1 })
db.user_activity.createIndex({ propertyId: 1, eventType: 1 })
db.user_activity.createIndex({ createdAt: 1 }, { expireAfterSeconds: 15552000 })   // 180 days
db.recommendations_cache.createIndex({ userId: 1 }, { unique: true })
db.recommendations_cache.createIndex({ generatedAt: 1 }, { expireAfterSeconds: 3600 })   // 1 hour
```

The validators above are condensed to the rules documented in the NoSQL component report (required fields, allowed event types, item structure). The full property-by-property definitions are in db/mongo-setup.js.

16.2 Inserting documents

```
db.user_activity.insertOne({ userId: NumberInt(5), propertyId: NumberInt(6), eventType: "VIEW",
  metadata: { source: "recommendations" }, createdAt: new Date() })

db.user_activity.insertOne({ userId: NumberInt(5), propertyId: null, eventType: "SEARCH",
  metadata: { filters: { city: "Bengaluru", listingType: "RENT", minPrice: 30000, maxPrice: 60000 } },
  createdAt: new Date() })

db.user_preferences.insertOne({ userId: NumberInt(5),
  preferences: { city: "Bengaluru", listingType: "RENT", minPrice: 30000, maxPrice: 60000, minBedrooms: 2,
                 preferredPropertyTypes: ["Apartment"], preferredAmenities: ["Parking","Lift"] },
  updatedAt: new Date() })
```

16.3 Queries run in the project database

Command	Purpose	Figure
`db.user_activity.findOne() / db.user_activity.find().limit(5)`	Inspect activity documents.	4
`db.user_preferences.find().limit(5)`	Inspect stored buyer preferences.	5
`db.recommendations_cache.find().limit(5)`	Inspect the cached recommendation document.	6, 7
`db.user_activity.getIndexes() / db.user_preferences.getIndexes() / db.recommendations_cache.getIndexes()`	List compound, unique and TTL indexes.	8
`db.user_activity.countDocuments() / db.user_preferences.countDocuments() / db.recommendations_cache.countDocuments()`	Count documents: 201, 2, 1.	18
`db.user_activity.distinct("eventType")`	List the event types present.	19
`db.user_activity.aggregate([ { $group: { _id: null, totalActivities: { $sum: 1 } } } ])`	Total activities (201).	20
`db.user_activity.aggregate([ { $group: { _id: "$eventType", total: { $sum: 1 } } }, { $sort: { total: -1 } } ])`	Activities per event type, largest first.	20
`db.user_activity.find({ eventType: "VIEW" }).limit(5)`	Filter events by type.	21
`db.user_activity.find({ userId: 5 })`	One buyer’s activity history.	22
`db.recommendations_cache.findOne({ userId: 5 })`	Read one buyer’s cached recommendations.	—

16.4 Update and delete commands

```
// Refresh a buyer's cached recommendations (one cache document per user, enforced by the unique index)
db.recommendations_cache.replaceOne({ userId: 5 }, { userId: 5, items: [ /* ranked items */ ], generatedAt:
    new Date() }, { upsert: true })

// Save changed preferences
db.user_preferences.updateOne({ userId: 5 }, { $set: { budgetMax: 65000, updatedAt: new Date() } })

// Remove a buyer's activity (TTL already removes events older than 180 days)
db.user_activity.deleteMany({ userId: 5, eventType: "SEARCH" })
```

Vector database commands: not applicable

PropMatch does not use embeddings, a vector index or nearest-neighbour search, so there are no vector commands to list. See Section 11.5.

17 Evaluation Rubrics – Completion Status

Status key: **Completed**, **Partial**, **Pending**, **Not applicable**.

17.1 Submission contents

#	Item	Status	Where / note
1	Project title	Completed	Cover page and Project Details
2	Student SRNs and names	Completed	Cover page and Project Details
3	Abstract	Completed	Page 3
4	Technology stack	Completed	Section 4
5	SQL – database commands	Completed	Section 15 (DDL, DML) and Sections 10, 12
6	NoSQL / vector database commands	Partial	NoSQL completed in Section 16; vector commands not applicable (vector search not implemented)
7	Next.js – project tree structure	Pending	Needs the project folder listing
8	Sample screenshots of application code and demo	Partial	Demo screenshots completed (Section 13); code screenshots pending
9	Evaluation rubrics – completion status	Completed	This section

17.2 Project documentation checklist

#	Documentation item	Status	Section
1	Objectives	Completed	2
2	Functional requirements	Completed	3
3	ER diagram	Completed	6
4	Relational schema	Completed	7
5	Database choice and justification	Completed	5
6	Data dictionary / schema description	Completed	8, 9
7	SQL implementation	Completed	10, 15
8	NoSQL implementation	Completed	11, 16
9	Vector database implementation	Not applicable	11.5 (not implemented)
10	Important queries	Completed	12
11	System architecture	Completed	4
12	Application screenshots	Completed	13
13	Testing results	Completed	14
14	Conclusion	Completed	18

17.3 Technical features

Feature	Status	Evidence
Relational design: 13 tables, keys and relationships	Completed	Sections 6–8; constraints verified (Figure 36)
Integrity constraints (PK, FK, UNIQUE, CHECK, DEFAULT, ENUM)	Completed	43 constraints; tests DB-01 to DB-12
Triggers, function, stored procedures	Completed	Section 10; tests DB-13 to DB-28
Views	Completed	v_active_listings, v_agent_performance; tests DB-29, DB-30
SQL operations (SELECT, WHERE, ORDER BY, DISTINCT, GROUP BY, COUNT, HAVING, JOIN, INSERT, UPDATE, DELETE)	Completed	Sections 12 and 15
MongoDB collections, indexes and TTL retention	Completed	Section 11; Figures 4–8, 18–22
MongoDB validation	Partial	Validators in warn mode; some stored preference documents do not yet match (Section 11.4)
Personalised recommendations with match score and reasons	Completed	Figures 6, 7 and 30
Vector embeddings and vector search	Not applicable	Not implemented (Section 11.5)
Web application: buyer and agent screens	Completed	Section 13
Database testing	Completed	32 of 32 tests passed (Section 14)
Automated end-to-end application tests	Partial	Not supplied; application behaviour evidenced by screenshots and cross-checks

This status is based on the documentation checklist and the nine submission items. If your course provides a marked evaluation rubric with its own criteria and marks, the rows can be replaced with its exact wording.

18 Advantages, Conclusion and Future Enhancements

Advantages

- Centralized storage of property information.
- Relationships between properties, users, agents and locations.
- Reduced duplication through normalized related tables.
- Improved consistency through constraints, ENUM columns, triggers and transactional procedures.
- Easier retrieval and filtering of property listings, backed by purpose-built indexes.
- Support for inquiry, favourites and site-visit records.
- Access to summarized information through database views.
- Flexible buyer preference and activity storage in MongoDB, with automatic expiry through TTL indexes.
- Explainable recommendations: every suggestion carries a match score and reasons.

Conclusion

PropMatch demonstrates the use of relational and NoSQL database concepts together in a property listing and management application. The MySQL database provides structured tables, primary and foreign keys, 43 integrity constraints, triggers, stored procedures, a function, joins, aggregate queries and views. The MongoDB database adds three collections for buyer preferences, activity events and cached recommendations, with validators, unique and compound indexes and TTL retention. A Next.js application exposes both to buyers and agents.

The database rules were verified by 32 executed tests that all passed, and the live databases and application screens are consistent with each other. Vector embeddings and semantic search are not part of the current implementation. The project provides a solid foundation for managing property information and related user activity; its overall completeness depends on continued integration testing of the frontend, backend and authentication layers.

Future enhancements

#	Enhancement
1	Improve property search and filtering, including full-text search in the interface.
2	Add more detailed agent and property analytics.
3	Strengthen role-based access control, including an admin workflow to verify agents.
4	Improve automated testing and API validation, including end-to-end browser tests.
5	Extend reporting and property recommendation features.
6	Add vector embeddings and semantic search to MongoDB, documenting the model, dimensions, index and similarity metric.
7	Align all MongoDB documents with their validators, then switch validationAction from warn to error.
8	Exclude cancelled visits from v_agent_performance totals if only active visits should count.